

Capítulo 04 — Sección 01

¿Por qué la memoria cambió la IA moderna?

En el capítulo anterior identificamos la memoria como una de las cuatro estrategias para superar los límites de la ventana de contexto. La nombramos, la catalogamos junto al resumen, la comprensión y la recuperación externa, y avanzamos. En este capítulo la convertimos en una disciplina de diseño.

Ese cambio de perspectiva no es cosmético. Durante los primeros años de los modelos de lenguaje de gran escala, la forma dominante de pensar la interacción era la sesión única: el usuario escribía, el modelo respondía, la conversación terminaba. Si el mismo usuario volvía al día siguiente, el modelo lo recibía como a un extraño. No había continuidad, no había contexto acumulado, no había ninguna forma de que el sistema aprendiera —incluso en el sentido más básico de la palabra— quién era ese usuario, qué había necesitado antes, o qué patrones definían su forma de trabajar.

Este modelo de sesión única fue suficiente mientras las aplicaciones de IA eran herramientas ocasionales. Pero cuando las aplicaciones comenzaron a integrarse como asistentes permanentes —en flujos de trabajo, en plataformas empresariales, en agentes autónomos que ejecutan tareas durante horas o días—, la ausencia de memoria dejó de ser una limitación técnica menor y se convirtió en un bloqueante arquitectónico.

El salto que cambia la ecuación

Consideremos dos sistemas hipotéticos resolviendo el mismo problema: asistir a un analista financiero durante su jornada de trabajo.

El **sistema sin memoria** recibe cada consulta de forma aislada. Cuando el analista pregunta "¿cómo debería estructurar el informe de riesgo para el cliente del sector energético?", el sistema responde con una estructura genérica. Cuando, una hora después, pregunta "¿puedes ajustar el tono para que sea más conservador?", el sistema no sabe a qué informe se refiere, qué cliente es, qué estructura se acordó ni qué significa "más conservador" en ese contexto. Cada respuesta empieza desde cero.

El **sistema con memoria** recuerda que el analista trabaja con tres clientes del sector energético, que el cliente más reciente prefiere informes sintéticos con métricas primero y

narrativa después, que en el último informe el analista eligió una paleta de colores específica para los gráficos de riesgo, y que la palabra "conservador" en este contexto ha significado históricamente reducir la exposición proyectada en un 15%. La segunda consulta, en este sistema, produce un ajuste quirúrgico en lugar de una respuesta genérica.

La diferencia entre ambos sistemas no está en el modelo subyacente. El modelo de lenguaje es idéntico. La diferencia está en la arquitectura de memoria que envuelve al modelo.

Por qué esto cambió la IA moderna

Tres factores convergieron para hacer de la memoria una prioridad de diseño:

La proliferación de agentes de larga duración. A medida que los sistemas de IA comenzaron a ejecutar tareas que se extienden en el tiempo —investigación, gestión de proyectos, automatización de procesos—, la necesidad de mantener coherencia entre sesiones se volvió imposible de ignorar. Un agente que no recuerda qué investigó ayer no puede construir sobre ese trabajo hoy.

La expectativa de personalización. Los usuarios que interactúan con un sistema de IA durante semanas desarrollan la expectativa razonable de que el sistema los conoce. No en el sentido de vigilancia o acumulación invasiva de datos, sino en el sentido de que un buen colaborador humano también recuerda las preferencias de quien trabaja con él. Cuando esa expectativa no se cumple, la percepción de inteligencia del sistema cae drásticamente, independientemente de la calidad del modelo.

La imposibilidad de meter todo en la ventana de contexto. Ya exploramos este problema en los capítulos anteriores. El contexto tiene un límite. La memoria es la respuesta de ingeniería a ese límite: en lugar de intentar meter todo en el prompt, diseñamos un sistema que sabe qué recuperar y cuándo.

La memoria como disciplina de diseño

Lo que hace especial al diseño de memoria en IA no es la tecnología de almacenamiento —las bases de datos existen desde hace décadas— sino las decisiones de diseño que rodean al almacenamiento: qué guardar, con qué granularidad, durante cuánto tiempo, cómo recuperarlo, cómo decidir cuándo algo debe ser olvidado.

Estas decisiones tienen consecuencias directas sobre la calidad de las respuestas, la privacidad de los usuarios, el costo operativo del sistema y su comportamiento a largo plazo. Un sistema que guarda demasiado se vuelve ruidoso y caro. Un sistema que guarda demasiado poco es indistinguible de uno sin memoria. Un sistema que guarda las cosas equivocadas puede producir respuestas sesgadas o desactualizadas.

El diseño de memoria no es un problema de almacenamiento. Es un problema de criterio.

En las próximas secciones construiremos ese criterio desde los fundamentos: primero desde la analogía cognitiva que nos da el marco conceptual, luego desde la arquitectura que nos da la estructura técnica, y finalmente desde los patrones y casos reales que nos dan la práctica.

La siguiente sección examina la memoria humana como modelo conceptual para el diseño de memorias en IA. No porque la IA replique la cognición humana —no lo hace—, sino porque la taxonomía cognitiva nos da un vocabulario preciso para distinguir tipos de información que los sistemas de IA necesitan gestionar de formas radicalmente distintas.

Capítulo 04 — Sección 02

La memoria humana como inspiración

La psicología cognitiva distingue varios sistemas de memoria en los seres humanos, cada uno con características propias de almacenamiento, duración y función. Esta distinción no es solo académica: cuando empezamos a diseñar sistemas de IA que necesitan recordar, descubrimos que la taxonomía cognitiva mapea con sorprendente precisión sobre los problemas de arquitectura que tenemos que resolver.

No estamos diciendo que la IA funciona como el cerebro humano. No lo hace. Estamos usando la taxonomía cognitiva como andamiaje conceptual porque es una de las pocas formas que tenemos de nombrar con precisión las diferencias entre tipos de información que los sistemas de IA necesitan gestionar de maneras fundamentalmente distintas.

Los cuatro tipos de memoria y su equivalente en IA

Memoria de trabajo

En la cognición humana, la memoria de trabajo es el espacio mental activo donde procesamos la información en este momento. Tiene capacidad limitada —se estima entre 4 y 7 elementos simultáneos—, se volatiliza rápidamente cuando no se le presta atención, y es el escenario de todo razonamiento consciente.

En los sistemas de IA, el equivalente exacto es la **ventana de contexto**. Todo lo que el modelo puede "pensar" en este momento está dentro de esa ventana. Tiene un límite de tokens. Su contenido desaparece cuando termina la sesión. Y es el lugar donde ocurre el razonamiento del modelo.

Este equivalente no es metafórico: es funcional. Cuando diseñamos la ventana de contexto —qué incluir, en qué orden, con qué formato— estamos literalmente diseñando la memoria de trabajo del sistema.

Memoria episódica

La memoria episódica almacena eventos específicos con su contexto temporal: "la reunión del martes", "la llamada de ayer con el cliente", "lo que acordamos en el sprint anterior". Es

personal, situada en el tiempo, y permite reconstruir secuencias de eventos.

En los sistemas de IA, la **memoria conversacional persistida** cumple esta función. Son los registros de interacciones pasadas que el sistema puede consultar para entender la historia de su relación con el usuario: qué se discutió, qué decisiones se tomaron, qué problemas surgieron. Una base de datos de conversaciones estructuradas, un historial de acciones ejecutadas por un agente, un log de tareas completadas —todos son formas de memoria episódica artificial.

La característica clave es que estos registros son **situados en el tiempo y en el evento**. No es conocimiento general: es "esto pasó, en este contexto, en este momento".

Memoria semántica

La memoria semántica almacena conocimiento general sobre el mundo: hechos, conceptos, relaciones entre ideas. No está ligada a un evento específico —no recordamos cuándo aprendimos que París es la capital de Francia—, sino que representa conocimiento descontextualizado y disponible para su uso general.

En los sistemas de IA, la **memoria semántica** corresponde al conocimiento estructurado sobre el dominio, el usuario o la organización que el sistema acumula a través del tiempo. No es "en la sesión del martes el usuario dijo X" sino "el usuario trabaja en el sector financiero, prefiere respuestas sintéticas, y tiene expertise en derivados de renta fija". Es el perfil destilado, el conocimiento factual sobre el dominio de aplicación, las reglas de negocio aprendidas.

Esta es la forma de memoria con mayor valor a largo plazo y la que más cuidado requiere en términos de actualización y mantenimiento.

Memoria procedimental

La memoria procedimental almacena cómo se hacen las cosas: habilidades, procedimientos, rutinas. Es la memoria que activa el pianista cuando toca sin pensar en cada nota, o la del conductor que cambia de marcha sin reconstruir el proceso consciente.

En los sistemas de IA, la **memoria procedimental** se implementa de varias formas: como instrucciones de sistema que definen el comportamiento esperado, como flujos de herramientas guardados que el agente reutiliza, o como plantillas y estructuras de respuesta que el sistema aplica automáticamente a ciertos tipos de solicitudes. Es el "cómo actúa" del sistema, no el "qué sabe".

Mapeando la taxonomía a la arquitectura

TIPO DE MEMORIA	DURACIÓN	CONTENIDO	EQUIVALENTE EN IA
Memoria de trabajo	Segundos/min	Información activa	Ventana de contexto
Memoria episódica	Días/años	Eventos y conversac.	Historial persistido
Memoria semántica	Permanente	Conocimiento factual	Perfiles y knowledge ba
Memoria procedum.	Permanente	Procedimientos	System prompts y flujos

Este mapa tiene implicancias de diseño directas. Cuando un ingeniero de IA decide qué información guardar y dónde, está respondiendo implícitamente a la pregunta: ¿qué tipo de memoria es esta? Un evento de conversación va a memoria episódica. Una preferencia estable del usuario va a memoria semántica. Una instrucción de comportamiento va a memoria procedimental. El contexto activo de la sesión actual va a la ventana de trabajo.

Confundir estas categorías produce sistemas con problemas predecibles: sistemas que guardan todo en el contexto activo (y se quedan sin tokens), sistemas que no distinguen entre hechos temporales y permanentes (y contaminan la memoria semántica con datos caducos), o sistemas que no tienen memoria procedimental explícita (y producen comportamientos inconsistentes entre sesiones).

Una nota sobre el límite de la analogía

La analogía cognitiva es útil como marco, pero tiene límites que el ingeniero debe tener presentes.

La memoria humana es reconstructiva —cuando recordamos, reinterpretemos y completamos—, mientras que la memoria artificial es, en principio, literal: recuperamos lo que guardamos. Esto es una ventaja (precisión) y un riesgo (sin criterio de qué guardar, guardamos ruido con la misma fidelidad que guardamos señal).

La memoria humana olvida adaptativamente —los recuerdos que no se activan se degradan—, mientras que la memoria artificial persiste indefinidamente a menos que diseñemos mecanismos explícitos de degradación o eliminación. Este aspecto —el olvido como función de diseño— lo desarrollaremos en detalle en la sección 07.

La memoria humana tiene un cuerpo, un sistema emocional y un contexto social que modulan qué se recuerda. Los sistemas de IA no tienen ninguna de estas señales naturales de relevancia. Por eso el ingeniero debe diseñar criterios explícitos de relevancia y prioridad donde el cerebro humano los tiene implícitamente.

La siguiente sección presenta la arquitectura general de memoria en sistemas de IA: cómo se organizan los componentes, cómo fluye la información entre ellos, y qué decisiones de diseño determinan si un sistema de memoria funciona bien en producción.

Capítulo 04 — Sección 03

Arquitectura general de memoria

Un sistema de memoria en IA no es una base de datos con un modelo de lenguaje conectado. Es un conjunto de componentes que trabajan de forma coordinada para capturar, organizar, almacenar, recuperar e inyectar información en el momento correcto. Cuando uno de estos componentes falla o no existe, el sistema entero produce comportamientos degradados que suelen diagnosticarse erróneamente como problemas del modelo.

Esta sección presenta la arquitectura completa. Las secciones siguientes desarrollan cada componente con profundidad técnica.

| Los cinco componentes de un sistema de memoria

Un sistema de memoria bien diseñado tiene cinco componentes funcionales:

```

[ENTRADA / CONVERSACIÓN]

|
v

[1. CAPTURA]

¿Qué vale la pena recordar?

|
v

[2. PROCESAMIENTO]

¿Cómo se estructura y clasifica?

|
v

[3. ALMACENAMIENTO]

¿Dónde y con qué formato?

|
----+----
|      |
v      v

[Episódico] [Semántico] [Procedimental]

|      |
----+----

|
v

[4. RECUPERACIÓN]

¿Qué es relevante para esta consulta?

|
v

[5. INYECCIÓN]

¿Cómo se incorpora al contexto activo?

|
v

[MODELO DE LENGUAJE / RESPUESTA]

```

Componente 1: Captura

La captura es el momento en que el sistema decide qué información de la interacción actual merece ser almacenada para uso futuro. Este componente es el más crítico y el más

descuidado en implementaciones ingenuas.

Una captura indiscriminada —guardar todo— produce bases de memoria ruidosas, costosas de consultar y propensas a recuperar información irrelevante. Una captura demasiado restrictiva produce sistemas que no aprenden nada útil de la interacción.

Los criterios de captura deben responder a preguntas como:

- ¿Esta información es relevante en conversaciones futuras o solo en esta sesión?
- ¿Es un hecho factual sobre el usuario, el dominio o la aplicación?
- ¿Hay una preferencia, restricción o acuerdo que el sistema debería recordar?
- ¿Esta información contradice o actualiza algo ya almacenado?

La captura puede ser **explícita** —el usuario o el agente indica deliberadamente que algo debe recordarse— o **implícita** —el sistema infiere qué vale la pena guardar basándose en criterios predefinidos.

Componente 2: Procesamiento

Antes de almacenar, la información capturada necesita ser procesada: clasificada por tipo (episódica, semántica, procedimental), normalizada en formato, y potencialmente fusionada con registros existentes.

El procesamiento también incluye la **extracción de entidades y relaciones**: si el usuario menciona "mi cliente del sector energético en Buenos Aires", el procesamiento extrae "cliente", "sector energético" y "Buenos Aires" como entidades relacionadas, en lugar de guardar la frase literal. Esta representación estructurada habilita recuperaciones más precisas.

En sistemas avanzados, el procesamiento incluye un paso de **resolución de conflictos**: si la memoria existente dice que el usuario trabaja en el sector bancario y la nueva interacción dice que trabaja en energía, el sistema debe resolver la contradicción, no acumularla.

Componente 3: Almacenamiento

El almacenamiento no es un componente único sino una capa con múltiples backends según el tipo de memoria:

- **Almacenamiento de memoria episódica**: bases de datos relacionales o documentales que registran eventos, conversaciones e interacciones con sus metadatos temporales (timestamp, sesión, usuario, herramientas usadas).

- **Almacenamiento de memoria semántica:** bases de datos vectoriales para recuperación por similitud semántica, bases de datos key-value para perfiles estructurados, o grafos de conocimiento para representar relaciones complejas entre entidades.
- **Almacenamiento de memoria procedimental:** generalmente archivos de configuración, system prompts versionados o repositorios de plantillas. En agentes avanzados, puede incluir bases de datos de flujos de trabajo.

La elección del backend depende de cómo se va a recuperar la información. Si la recuperación será por búsqueda exacta de un campo (el nombre del cliente, el ID de un proyecto), un key-value store es suficiente. Si la recuperación será por similitud conceptual ("algo parecido a lo que preguntó antes"), se necesita una base de datos vectorial. Si las relaciones entre entidades son parte del modelo de datos, un grafo de conocimiento puede ser la opción correcta.

Componente 4: Recuperación

La recuperación es el proceso de encontrar, entre todo lo almacenado, lo que es relevante para la consulta o tarea actual. Es el componente que determina si el sistema produce respuestas contextualizadas o respuestas genéricas, aunque la memoria esté bien poblada.

Las estrategias de recuperación incluyen:

- **Recuperación por clave:** se sabe exactamente qué buscar (el perfil del usuario con ID específico, el resumen de la última sesión).
- **Recuperación semántica:** se busca por similitud con la consulta actual. Requiere embeddings y búsqueda vectorial.
- **Recuperación por tiempo:** se recuperan los N registros más recientes, o los registros dentro de una ventana temporal.
- **Recuperación híbrida:** combinación de criterios, por ejemplo, los registros más recientes y semánticamente más similares a la consulta actual.

Un sistema de recuperación mal diseñado puede recuperar información desactualizada, información de baja relevancia, o directamente no encontrar información relevante que sí existe en el almacenamiento.

Componente 5: Inyección

La inyección es el acto de incorporar la memoria recuperada al contexto activo del modelo. No es trivial: la información recuperada compite por espacio en la ventana de contexto con las

instrucciones del sistema, el historial de la conversación actual, los resultados de herramientas y la consulta del usuario.

Las decisiones de inyección incluyen:

- ¿Dónde en el contexto se coloca la memoria recuperada? (al inicio del system prompt, como bloque separado, intercalada con el historial)
- ¿En qué formato? (texto plano, JSON estructurado, bullets)
- ¿Cuánta memoria se inyecta? (los 3 registros más relevantes, los últimos 5 eventos, el perfil completo del usuario)
- ¿Cómo se indica al modelo qué es memoria y qué es contexto actual?

Una inyección bien diseñada hace que el modelo use la memoria sin esfuerzo. Una inyección mal diseñada produce confusión entre lo que pasó antes y lo que está pasando ahora, o simplemente consume tokens sin mejorar la respuesta.

El ciclo completo en una interacción real

Para concretar la arquitectura, trazamos el ciclo completo en un ejemplo:

Un usuario de un asistente de análisis financiero pregunta: "¿Puedes preparar un borrador del informe de riesgo para este trimestre?"

1. **Captura previa:** En conversaciones anteriores, el sistema capturó que este usuario trabaja con tres fondos de inversión, que prefiere informes con resumen ejecutivo al inicio, y que el trimestre anterior señaló exposición al riesgo cambiario como prioridad.
 2. **Recuperación:** Al recibir la nueva consulta, el sistema activa la recuperación semántica con "informe de riesgo" como ancla. Recupera: el perfil del usuario, el template de informe aprobado en la sesión anterior, y el registro del comentario sobre riesgo cambiario.
 3. **Inyección:** Los tres elementos recuperados se incorporan al contexto activo como bloque de "contexto del usuario" antes del historial de la sesión actual.
 4. **Respuesta:** El modelo genera un borrador que incluye la estructura preferida, los fondos relevantes y menciona el riesgo cambiario como ítem de atención prioritaria.
 5. **Captura posterior:** El sistema registra que el usuario solicitó un informe de riesgo en este trimestre, qué draft se generó, y si el usuario lo aprobó o lo modificó. Este registro alimenta la memoria episódica para la próxima sesión.
-

La siguiente sección profundiza en la memoria conversacional: el tipo de memoria más cercano a la ventana de contexto, sus estrategias de gestión, y las decisiones de diseño que determinan cuánto del historial de una conversación vale la pena mantener activo.

Capítulo 04 — Sección 04

Memoria conversacional

La memoria conversacional es la forma más inmediata de memoria en un sistema de IA: es el registro de lo que ocurrió en la conversación actual. En los modelos de lenguaje, esa memoria existe naturalmente dentro de la ventana de contexto: mientras el historial de mensajes cabe en el contexto, el modelo tiene acceso a todo lo que se dijo.

El problema, como ya vimos en el módulo anterior, es que la ventana de contexto tiene un límite. Una conversación larga, con muchas preguntas y respuestas extensas, eventualmente supera ese límite. En ese punto, algo tiene que ceder: o se trunca el historial, o se comprime, o se diseña una estrategia deliberada de gestión.

Esta sección examina las estrategias de gestión de memoria conversacional, sus trade-offs y cuándo elegir cada una.

Estrategia 1: Historial completo

La estrategia más simple es mantener el historial completo de la conversación dentro del contexto. Cada turno de la conversación —mensaje del usuario y respuesta del modelo— se agrega secuencialmente.

```
CONTEXTO = [system_prompt] + [turno_1] + [turno_2] + ... + [turno_N]
```

Ventajas:

- El modelo tiene acceso a toda la información de la conversación sin pérdida.
- La coherencia de la conversación es máxima.
- Sin costo adicional de procesamiento (no hay pasos intermedios de compresión o recuperación).

Limitaciones:

- El costo de tokens crece linealmente con la longitud de la conversación.
- Cuando se alcanza el límite de contexto, el sistema necesita truncar o la conversación falla.
- En conversaciones muy largas, el modelo puede tener dificultades para enfocarse en la

información más reciente cuando está "enterrada" bajo mucho contexto anterior (el problema conocido como "lost in the middle").

Cuándo usar esta estrategia:

- Conversaciones cortas y acotadas (soporte técnico de primer nivel, consultas de información simple).
- Aplicaciones donde la conversación no se extiende más de 10-20 turnos.
- Contextos donde la precisión absoluta es crítica y cualquier pérdida de información es inaceptable.

Estrategia 2: Ventana deslizante

La ventana deslizante mantiene solo los últimos N turnos de la conversación en el contexto. Cuando se agrega un nuevo turno, el turno más antiguo se elimina.

```
CONTEXTO = [system_prompt] + [turno_(N-k+1)] + ... + [turno_N]
```

Ventajas:

- El tamaño del contexto se mantiene controlado y predecible.
- El modelo siempre trabaja con la información más reciente.
- El costo de tokens es constante (no crece con la duración de la conversación).

Limitaciones:

- Los turnos que salen de la ventana se pierden por completo.
- Si se mencionó algo importante en el turno 3 y ahora estamos en el turno 25, esa información ya no existe para el modelo.
- Puede producir respuestas que ignoran contexto crítico establecido al inicio de la conversación.

Cuándo usar esta estrategia:

- Conversaciones de soporte donde cada consulta es relativamente autónoma.
- Aplicaciones donde el usuario rara vez necesita hacer referencia a información de muchos turnos atrás.
- Situaciones donde el costo es una restricción más importante que la completitud.

Nota del arquitecto: Una variante más refinada es la ventana deslizante con anclaje: además de los N turnos recientes, el sistema siempre incluye el primer turno (donde generalmente se establece el objetivo principal de la conversación) y cualquier turno marcado

explícitamente como importante. Esta variante reduce significativamente el problema de pérdida de contexto inicial.

Estrategia 3: Resumen progresivo

En lugar de descartar los turnos que salen de la ventana, el sistema los resume. El resumen reemplaza al historial completo como representación comprimida del contexto pasado.

```
CONTEXTO = [system_prompt] + [resumen_de_turnos_anteriores] + [turno_reciente_1] + ...
```

Ventajas:

- Se preserva información de toda la conversación, aunque de forma comprimida.
- El tamaño del contexto se mantiene manejable.
- Los temas, acuerdos y decisiones clave sobreviven en el resumen.

Limitaciones:

- El proceso de resumen tiene costo (un LLM call adicional, o computación del proceso de summarización).
- Introduce latencia.
- El resumen puede perder detalles que luego resultan importantes.
- La calidad del resumen afecta directamente la calidad de las respuestas posteriores.

Cuándo usar esta estrategia:

- Conversaciones largas donde la coherencia a largo plazo es importante.
- Sesiones de trabajo extendidas (brainstorming, redacción colaborativa, análisis iterativo).
- Aplicaciones donde el usuario puede referirse a cosas dichas "antes" de forma indeterminada.

Implementación práctica:

El resumen puede generarse de dos formas:

Resumen por umbral: cuando el historial supera un número de tokens definido, se llama al LLM para que genere un resumen del historial anterior y se reemplaza ese historial con el resumen.

```
def gestionar_historial(historial, umbral_tokens=3000):
    tokens_actuales = contar_tokens(historial)

    if tokens_actuales > umbral_tokens:
        turnos_a_resumir = historial[:-4] # Conservar los últimos 4 turnos
        turnos_recientes = historial[-4:]

        resumen = generar_resumen(turnos_a_resumir)

        return [{"role": "system", "content": f"Resumen de la conversación anterior: {resumen}"}]
    return historial
```

Resumen incremental: después de cada turno, el sistema actualiza el resumen acumulativo. Es más costoso pero produce resúmenes más coherentes.

Estrategia 4: Memoria conversacional híbrida

La estrategia más robusta para aplicaciones de producción combina las tres anteriores:

- Los últimos N turnos se mantienen como historial completo (máxima fidelidad para el contexto inmediato).
- Los turnos más antiguos se comprimen en un resumen (preservación de coherencia).
- Los elementos marcados como críticos (acuerdos, restricciones, objetivos) se extraen y almacenan en la memoria semántica persistente (disponibles en sesiones futuras).

```
CONTEXT = [system_prompt]
CONTEXT += [memoria_semantica_del_usuario]  ← recuperada del almacenamiento persistente
CONTEXT += [resumen_de_turnos_anteriores]   ← generado en sesión
CONTEXT += [historial_reciente]             ← últimos N turnos completos
```

Esta estrategia es la que usan los sistemas de agentes más maduros. Su costo es mayor —en latencia y en tokens—, pero es la única que combina coherencia a corto plazo, preservación a largo plazo y personalización entre sesiones.

Comparación de estrategias

ESTRATEGIA	COHERENCIA	COSTO	COMPLEJIDAD	CASO DE USO TÍPICO
-----	-----	-----	-----	-----
Historial completo	Alta	Alto	Baja	Chats cortos
Ventana deslizante	Media	Bajo	Baja	Soporte básico
Resumen progresivo	Media-Alta	Medio	Media	Sesiones largas
Híbrida	Alta	Alto	Alta	Agentes de producción

La siguiente sección aborda la memoria persistente: el mecanismo que permite que el sistema recuerde información entre sesiones distintas, construyendo un modelo acumulativo del usuario y del dominio de trabajo.

Capítulo 04 — Sección 05

Memoria persistente

La memoria conversacional existe dentro de una sesión. Cuando la sesión termina, esa memoria desaparece. La memoria persistente es la respuesta a esa discontinuidad: información almacenada fuera del modelo, en un sistema externo de almacenamiento, disponible para ser recuperada en cualquier sesión futura.

Si la memoria conversacional es la memoria de trabajo de la sesión actual, la memoria persistente es el cuaderno que el ingeniero deja sobre el escritorio antes de apagar la computadora: al día siguiente, todo lo importante sigue ahí.

Qué se persiste y qué no

No toda la información de una conversación merece ser persistida. Persistir indiscriminadamente produce bases de datos ruidosas y recuperaciones imprecisas. El primer principio del diseño de memoria persistente es la selectividad.

La información que típicamente vale la pena persistir incluye:

Preferencias del usuario:

- Formato de respuesta preferido (detallado vs. sintético, con ejemplos vs. sin ejemplos)
- Idioma o registro lingüístico
- Áreas de expertise o conocimiento previo que el sistema debería asumir
- Temas o enfoques que el usuario ha rechazado explícitamente

Hechos sobre el dominio de trabajo:

- Nombres de proyectos, clientes, productos o entidades recurrentes
- Restricciones o reglas de negocio mencionadas por el usuario
- Decisiones tomadas en sesiones anteriores que afectan el trabajo actual
- Estado actualizado de proyectos en curso

Contexto de la relación:

- Objetivos de largo plazo expresados por el usuario
- Historial de tareas completadas o en progreso
- Feedback dado sobre respuestas anteriores del sistema

La información que generalmente no vale la pena persistir incluye: preguntas de exploración sin conclusión, contenido trivial o rutinario, información válida solo para el contexto inmediato de una sesión específica.

Tecnologías de almacenamiento persistente

La elección del backend de almacenamiento depende de cómo se va a recuperar la información. Hay tres familias principales:

Key-value stores y bases de datos documentales

Son la opción más simple y apropiada para perfiles de usuario estructurados y memoria semántica de baja complejidad.

```
{
  "user_id": "usr_4821",
  "perfil": {
    "nombre": "Laura",
    "rol": "Analista de riesgo",
    "empresa": "Fondo Austral",
    "preferencias": {
      "formato_respuesta": "sintético con bullet points",
      "nivel_detalle_técnico": "alto",
      "idioma_código": "Python"
    },
    "dominios_expertise": ["renta fija", "derivados", "riesgo cambiario"],
    "proyectos_activos": ["informe_riesgo_Q3_2026", "modelo_stress_test"]
  },
  "ultima_actualizacion": "2026-07-24T15:32:00Z"
}
```

Cuándo usar: cuando la recuperación es siempre por ID de usuario o entidad conocida, cuando la estructura de datos es predecible, cuando no se necesita búsqueda por similitud semántica.

Tecnologías: Redis, MongoDB, DynamoDB, PostgreSQL con JSONB, SQLite para proyectos pequeños.

Bases de datos vectoriales

Son la opción apropiada cuando la recuperación necesita ser semántica: no busco "el perfil del usuario 4821" sino "algo relacionado con lo que el usuario preguntó antes sobre estrategias de cobertura".

En una base de datos vectorial, cada registro de memoria se almacena junto con su embedding —una representación numérica de su contenido semántico—. La recuperación consiste en encontrar los registros cuyos embeddings son más similares al embedding de la consulta actual.

```
# Ejemplo de almacenamiento en base de datos vectorial

def guardar_memoria(contenido: str, metadata: dict, coleccion: str):

    embedding = generar_embedding(contenido)

    registro = {

        "id": generar_id(),

        "contenido": contenido,

        "embedding": embedding,

        "metadata": {

            "user_id": metadata["user_id"],

            "timestamp": datetime.now().isoformat(),

            "tipo": metadata["tipo"], # "episodica", "semantica", "procedimental"

            "fuente": metadata["fuente"] # "conversacion", "documento", "inferida"

        }

    }

    coleccion.insert(registro)


# Recuperación por similitud semántica

def recuperar_memoria_relevante(consulta: str, user_id: str, top_k: int = 5):

    embedding_consulta = generar_embedding(consulta)

    resultados = coleccion.buscar(

        embedding=embedding_consulta,

        filtros={"user_id": user_id},

        top_k=top_k

    )

    return resultados
```

Cuándo usar: cuando la recuperación debe ser por relevancia conceptual, cuando el volumen de memorias es alto y la búsqueda exacta no escala, cuando se necesita encontrar memorias relacionadas sin saber exactamente cuáles son.

Tecnologías: Pinecone, Weaviate, Qdrant, Chroma, pgvector (extensión de PostgreSQL).

Grafos de conocimiento

Son la opción más expresiva pero también la más compleja. Permiten representar relaciones entre entidades de forma que el sistema puede razonar sobre conexiones: "Laura trabaja en Fondo Austral → Fondo Austral tiene exposición al sector energético → el cliente del sector energético es TotalEnergies Argentina".

Cuándo usar: cuando las relaciones entre entidades son parte del modelo de datos (no solo los atributos individuales), cuando el sistema necesita hacer razonamiento multi-hop, cuando el dominio es complejo con muchas entidades interrelacionadas.

Tecnologías: Neo4j, Amazon Neptune, MemGraph, o implementaciones ad-hoc sobre grafos Python (NetworkX) para sistemas pequeños.

Estrategias de escritura

La escritura en memoria persistente puede ocurrir de tres maneras:

Escritura al cierre de sesión: el sistema procesa toda la conversación al final y extrae los elementos persistentes. Tiene la ventaja de contar con el contexto completo de la sesión para decidir qué guardar.

Escritura incremental: el sistema escribe en memoria después de cada turno o cada grupo de turnos. Útil para sesiones muy largas o cuando la información es urgente (un agente que necesita que otros agentes accedan a información generada en tiempo real).

Escritura bajo demanda: el sistema solo escribe en memoria cuando detecta explícitamente que algo merece ser recordado —porque el usuario lo indica, o porque el modelo lo infiere con alta confianza. Más selectiva, más difícil de implementar bien.

El problema de la consistencia

La memoria persistente introduce un problema que la memoria conversacional no tiene: la información puede volverse inconsistente con la realidad.

Un perfil de usuario guardado hace seis meses puede contener información desactualizada: el usuario cambió de empresa, el proyecto que estaba activo ya cerró, las preferencias de formato evolucionaron. Si el sistema recupera y usa esa información sin verificarla, puede producir respuestas basadas en realidades que ya no existen.

La gestión de la consistencia requiere diseño explícito:

- **Timestamps:** toda memoria persistida debería incluir cuándo fue capturada y cuándo fue usada por última vez.
- **Vencimiento:** algunos tipos de memoria deberían tener fechas de expiración automáticas (información de proyectos activos, por ejemplo).
- **Resolución de conflictos:** cuando la nueva información contradice la memoria existente, el sistema debe actualizar la memoria en lugar de acumular versiones contradictorias.
- **Confirmación periódica:** para memorias de alta importancia, el sistema puede incluir en la conversación una verificación implícita de que la información sigue siendo correcta.

Este aspecto —cuándo y cómo actualizar o eliminar memoria— es tan importante que tiene su propia sección en este capítulo: la sección 07, dedicada a consolidación y olvido.

La siguiente sección examina la memoria semántica desde el punto de vista de la recuperación por similitud, y establece la distinción entre memoria semántica gestionada por la aplicación y la recuperación RAG —que será el tema del capítulo 06.

Capítulo 04 — Sección 06

Memoria semántica y recuperación

En la sección anterior vimos cómo la memoria persistente permite al sistema recordar información entre sesiones. En esta sección nos concentramos en la forma más conceptualmente rica de esa memoria: la memoria semántica.

La memoria semántica, como vimos en la taxonomía cognitiva, almacena conocimiento general sobre el dominio, el usuario y la organización. No son eventos ("en la sesión del jueves el usuario preguntó X") sino hechos destilados ("el usuario trabaja en riesgo de crédito y tiene expertise en modelos paramétricos"). Es el conocimiento que el sistema acumula sobre su mundo de trabajo.

Esta sección también establece una distinción que es crítica para el diseño de sistemas de IA: la diferencia entre memoria semántica de aplicación y recuperación RAG. Ambas usan tecnologías similares, pero responden a problemas fundamentalmente distintos.

Qué es la memoria semántica en IA

La memoria semántica en un sistema de IA es el conjunto de representaciones de conocimiento que el sistema ha construido sobre el usuario, el dominio de aplicación y el contexto organizacional, a través de la acumulación e inferencia sobre interacciones pasadas.

Ejemplos concretos:

MEMORIA SEMÁNTICA DEL USUARIO:

- Trabaja como Product Manager en empresa de logística
- Sus análisis siempre incluyen KPIs de tiempo de entrega y NPS
- Prefiere comparaciones con benchmarks del sector
- Ha rechazado recomendaciones basadas en reingeniería de procesos
- Su empresa opera en Argentina, Brasil y Chile

MEMORIA SEMÁNTICA DEL DOMINIO:

- La empresa llama "último kilómetro" al segmento de entrega residencial
- "El cliente A" es su cuenta más grande, con contrato hasta diciembre 2026
- Los datos de logística tienen un lag de 48 horas antes de aparecer en el dashboard
- El equipo usa Notion para documentación y Jira para tracking de proyectos

Nótese que esta información no está en ningún manual ni base de conocimiento externa. Es conocimiento construido a través de las interacciones, que la aplicación gestiona activamente.

Cómo se construye la memoria semántica

La memoria semántica se construye mediante tres mecanismos:

Extracción directa: el sistema identifica afirmaciones factuales en las conversaciones y las almacena como hechos semánticos. "Trabajo en una empresa de logística con operaciones en tres países" se convierte en tres hechos estructurados: `industry: logistics`, `country_operations: [AR, BR, CL]`, `company_size: multinacional`.

Inferencia: el sistema infiere hechos a partir de patrones de comportamiento. Si el usuario siempre añade contexto de costos cuando el sistema no lo incluye, el sistema puede inferir `preferencia: incluir_análisis_de_costo = true`. Esta inferencia se puede almacenar con un nivel de confianza menor que los hechos explícitos.

Actualización: cuando la nueva información contradice la memoria existente, el sistema actualiza el hecho en lugar de crear un duplicado. Si el usuario menciona que cambió de empresa, los hechos relacionados con la empresa anterior se marcan como inactivos.

Recuperación semántica: embeddings y similitud

La recuperación de memoria semántica por similitud es el mecanismo que permite al sistema encontrar conocimiento relevante sin saber exactamente qué buscar.

El proceso tiene tres pasos:

1. Representación como vector (embedding):

Cada registro de memoria se convierte en un vector numérico de alta dimensión usando un modelo de embedding. Memorias con contenido semánticamente similar producen vectores que están cerca en ese espacio.

```
# "El cliente usa Python para análisis" → [0.21, -0.44, 0.87, ...]
# "Prefiere código en Python sobre R"   → [0.19, -0.41, 0.89, ...]
# "Trabaja en logística"                → [0.63,  0.12, 0.05, ...]
```

Los dos primeros vectores son cercanos (misma semántica). El tercero es distante.

2. Generación del vector de consulta:

Cuando llega una consulta ("¿Puedes mostrarme el análisis en código?"), se genera su embedding y se busca en la base de datos los registros cuyo vector es más cercano al vector de la consulta.

3. Ranking y selección:

Los registros más cercanos (mayor similitud coseno) se seleccionan y se inyectan en el contexto. El número de registros a seleccionar es un parámetro de diseño: demasiados satura el contexto, demasiado pocos puede perder información relevante.

La distinción crítica: memoria semántica vs. RAG

Esta distinción es fundamental y debe quedar completamente clara antes de continuar.

Dimensión	Memoria semántica	RAG (Retrieval-Augmented Generation)
Fuente del conocimiento	Construida por la aplicación a partir de interacciones	Documentos externos preexistentes
Quién la gestiona	La aplicación de IA	El sistema RAG (indexador + retriever)
Qué representa	Conocimiento sobre el usuario y el dominio de uso	Conocimiento sobre el mundo externo o la base de conocimiento de la organización
Ejemplo	"Este usuario prefiere respuestas sintéticas"	"El manual de producto dice que el componente X tiene una garantía de 2 años"
Cuando usar	Para personalizar la experiencia y recordar el contexto	Para responder preguntas sobre documentos, bases de conocimiento, o información que el modelo no tiene

La confusión entre ambos es común porque ambos usan bases de datos vectoriales y recuperación por similitud. La diferencia no está en la tecnología sino en el tipo de conocimiento y en quién lo produce.

La memoria semántica es conocimiento que la aplicación construye sobre su propio contexto de uso. RAG es conocimiento que la organización tiene en documentos y que el sistema recupera para responder preguntas.

Un sistema completo puede tener ambos. La memoria semántica le dice al sistema cómo hablar con este usuario específico. RAG le dice al sistema qué responder sobre el dominio de conocimiento de la organización.

El capítulo 06 está dedicado íntegramente a RAG. Lo que aquí establecemos es el punto de articulación: la memoria semántica es interna a la relación aplicación-usuario, RAG es externa y orientada al conocimiento del dominio.

Actualización de la memoria semántica

A diferencia de la memoria episódica —que simplemente acumula eventos— la memoria semántica requiere un mecanismo de actualización activa porque representa el estado actual del conocimiento, no una historia de eventos.

El proceso de actualización incluye:

Detección de contradicción: el sistema compara la nueva información con la memoria existente. Si encuentra una contradicción directa, no acumula ambas versiones —actualiza el hecho.

Versionado: en casos donde la trayectoria temporal importa, puede ser útil versionar los hechos en lugar de sobrescribirlos: `empresa = Fondo Austral (hasta 2025-12), empresa = LogisticAR (desde 2026-01)`.

Confianza decreciente: los hechos inferidos deberían tener un score de confianza que decae si no se confirman con evidencia adicional. Un hecho inferido hace 6 meses sin confirmación reciente debería tener menor peso en la recuperación.

La siguiente sección aborda uno de los problemas más importantes y menos discutidos del diseño de memoria: la consolidación y el olvido deliberado. Diseñar bien qué recordar es la mitad del trabajo; diseñar bien qué olvidar y cuándo es la otra mitad.

Capítulo 04 — Sección 07

Consolidación y olvido

Si solo diseñamos el lado de la captura y el almacenamiento, construimos sistemas de memoria que se degradan con el tiempo: acumulan ruido junto con la señal, mantienen información desactualizada con la misma prominencia que la información actual, y crecen sin control hasta que el costo de recuperación hace el sistema inviable.

El olvido deliberado no es una falla de diseño. Es una función de diseño. Los sistemas de memoria bien contruidos saben no solo qué recordar sino cuándo y cómo olvidar.

Esta sección distingue tres fenómenos distintos que a veces se confunden bajo el mismo nombre: el olvido catastrófico de los modelos de aprendizaje continuo, la caducidad natural de la información, y el olvido deliberado por diseño de políticas de retención.

El olvido catastrófico (y por qué no es nuestro problema principal aquí)

El olvido catastrófico es un fenómeno del entrenamiento de redes neuronales: cuando un modelo aprende nueva información, puede "sobreescribir" el conocimiento anterior de forma no controlada, degradando su desempeño en tareas previamente dominadas.

Es un problema activo de investigación en aprendizaje máquina, particularmente en escenarios de aprendizaje continuo (continual learning). Sin embargo, en el contexto del diseño de aplicaciones de IA que usan modelos preentrenados —que es el escenario de este libro—, el olvido catastrófico no aplica directamente: no estamos reentrenando el modelo, estamos diseñando el sistema de memoria externo que envuelve al modelo.

Lo mencionamos porque el término aparece frecuentemente en la literatura y puede crear confusión. El diseño de memorias de aplicación es un problema de ingeniería de sistemas, no de optimización de parámetros de red.

La caducidad natural de la información

Toda información tiene una vida útil. Algunos hechos son permanentes o cambian muy lentamente: el sector industrial de una empresa, el idioma del usuario, las preferencias de formato de respuesta. Otros hechos son altamente volátiles: el proyecto activo del mes, el precio de un activo financiero, el estado de una tarea.

Un sistema de memoria que no distingue entre estos tipos de información produce comportamientos extraños:

- El sistema recuerda que el proyecto "Alpha" está en progreso, aunque ese proyecto cerró hace tres meses.
- El sistema recuerda que el usuario trabaja en la empresa X, aunque cambió de trabajo.
- El sistema usa datos de precios recuperados de una sesión antigua como si fueran actuales.

La caducidad natural se gestiona con dos mecanismos:

TTL (Time To Live): cada registro de memoria tiene una fecha de expiración automática. Cuando se alcanza esa fecha, el registro se elimina o se marca como inactivo. El TTL apropiado depende del tipo de información.

TIPOS DE INFORMACIÓN Y TTL SUGERIDOS:

- Preferencias de formato del usuario	→ sin expiración (se actualiza por contradicción)
- Proyectos activos	→ 90 días desde última mención
- Precios, tasas, métricas cuantitativas	→ 24-48 horas
- Estado de tareas en progreso	→ 30 días
- Contactos y entidades nombradas	→ 180 días sin confirmación

Confianza decreciente: en lugar de eliminar abruptamente, el sistema puede reducir gradualmente el score de relevancia de un registro con el tiempo. Un hecho que no ha sido confirmado en meses tiene menos peso en la recuperación que uno confirmado recientemente.

El olvido deliberado: políticas de retención

El olvido deliberado va más allá de la caducidad automática: es el diseño explícito de políticas que determinan qué se guarda, durante cuánto tiempo y en qué condiciones.

Una política de retención bien diseñada responde a estas preguntas:

¿Qué entra a la memoria persistente?

Solo la información que supera un umbral de relevancia proyectada. No cada detalle de cada conversación, sino los elementos que tienen alta probabilidad de ser útiles en interacciones futuras.

¿Qué se consolida?

La consolidación es el proceso de comprimir múltiples memorias episódicas en una memoria semántica más compacta. En lugar de guardar diez registros que dicen "el usuario siempre añade contexto de costos", el sistema consolida esto en una sola entrada: `preferencia: incluir_análisis_de_costo = true, confianza: alta`.

La consolidación reduce el volumen de almacenamiento y mejora la precisión de la recuperación, pero requiere un proceso de inferencia que tiene costo computacional.

¿Qué se elimina?

- Información que ha sido explícitamente contradicha por información más reciente.
- Información que ha expirado su TTL sin confirmación.
- Información redundante (múltiples registros que dicen lo mismo).
- Información que el usuario ha solicitado eliminar (ver privacidad, más abajo).

¿Qué se archiva en lugar de eliminarse?

En algunos contextos —especialmente empresariales— la información no se elimina definitivamente sino que se archiva en un tier de almacenamiento de baja recuperabilidad. Esto permite auditoría sin contaminar la memoria activa.

Olvido solicitado por el usuario: privacidad y GDPR

El olvido deliberado tiene una dimensión de privacidad que el ingeniero no puede ignorar.

Los usuarios tienen el derecho —legalmente reconocido en muchas jurisdicciones bajo GDPR, LGPD y regulaciones similares— de solicitar que un sistema elimine sus datos. En el contexto de sistemas de memoria de IA, esto significa que la aplicación debe ser capaz de localizar y eliminar completamente toda la información persistida sobre un usuario, a pedido.

Las implicancias de diseño son significativas:

Toda memoria persistida debe estar identificada con el usuario al que pertenece.

Si la memoria está en una base de datos vectorial, cada vector debe tener metadatos de

`user_id` para que puedan ser encontrados y eliminados.

La eliminación debe ser completa. No solo el perfil de usuario, sino todos los registros episódicos, todas las preferencias inferidas, todos los resúmenes de sesión. Si el sistema usa caching, los datos en caché también deben ser invalidados.

El diseño debe prever esta operación desde el inicio. Añadir un mecanismo de eliminación a un sistema que no fue diseñado para ello es mucho más costoso que incluirlo en el diseño original.

```
def eliminar_memoria_usuario(user_id: str, colecciones: list[str]):  
    """  
    Elimina toda la memoria persistida de un usuario específico.  
    Opera sobre todas las colecciones de memoria del sistema.  
    """  
    registros_eliminados = 0  
    for coleccion in colecciones:  
        resultado = coleccion.eliminar(filtro={"user_id": user_id})  
        registros_eliminados += resultado.count  
        registrar_auditoria(  
            accion="eliminacion_por_solicitud",  
            user_id=user_id,  
            coleccion=coleccion.nombre,  
            registros=resultado.count,  
            timestamp=datetime.now()  
        )  
    return {"eliminados": registros_eliminados, "status": "completo"}
```

Nota del arquitecto: La conexión entre el diseño de memoria y la privacidad se desarrolla en mayor profundidad en el capítulo 14 (Seguridad y privacidad en sistemas de IA). Lo que aquí establecemos es la premisa de diseño: el olvido no es solo una función técnica de mantenimiento del sistema —es también un derecho del usuario que la arquitectura debe soportar.

La memoria compartida entre agentes y el olvido coordinado

En sistemas multiagente —que desarrollaremos en el capítulo 09—, la memoria puede ser compartida entre múltiples agentes que trabajan en paralelo o en secuencia. Esto introduce una complejidad adicional: el olvido de un agente puede afectar la coherencia de otros agentes que dependen de la misma información.

El diseño de políticas de retención en sistemas multiagente requiere un mecanismo de coordinación: antes de eliminar un registro, el sistema debe verificar que ningún agente activo tiene una dependencia sobre ese registro. Esto puede implementarse con contadores de referencia o con un registro centralizado de dependencias.

El principio general es que el olvido en un sistema multiagente debe ser coordinado, no unilateral.

La siguiente sección presenta las arquitecturas modernas de memoria que han emergido en los últimos años: frameworks como Memo, patrones de MemGPT, y las implementaciones que los equipos de producción usan en la actualidad.

Capítulo 04 — Sección 08

Arquitecturas modernas

Las secciones anteriores establecieron los principios del diseño de memoria: qué tipos de memoria existen, cómo se almacenan, cómo se recuperan, cómo se consolidan y cómo se olvidan. Esta sección conecta esos principios con las implementaciones concretas que están siendo adoptadas en producción.

Las arquitecturas que presentamos aquí no son experimentos de investigación. Son patrones que equipos de ingeniería están usando para construir sistemas con memoria persistente, y cuya evolución en los últimos dos años ha acelerado significativamente el estado del arte.

MemGPT: el modelo de gestión de memoria por capas

MemGPT es una arquitectura publicada en 2023 que introduce un modelo de gestión de memoria inspirado en los sistemas operativos: igual que un SO gestiona la memoria RAM y el almacenamiento en disco mediante paginación y swapping, MemGPT gestiona la ventana de contexto del LLM y el almacenamiento externo de manera análoga.

La idea central es que el LLM actúa como procesador que trabaja exclusivamente con lo que tiene en "memoria RAM" (la ventana de contexto), y cuando necesita información que no está en esa ventana, llama a una función para traerla desde el "almacenamiento externo" (la memoria persistente).



El LLM en MemGPT no recibe toda la memoria de golpe. Recibe solo lo que está en contexto, y cuando necesita más, llama explícitamente a las funciones de recuperación. Esto hace que el sistema sea escalable: la cantidad de memoria almacenada no está limitada por el tamaño del contexto.

Lo que MemGPT enseña al ingeniero: la memoria persistente debe ser accesible mediante funciones explícitas (herramientas), no inyectada masivamente al inicio de cada sesión. El LLM debe poder decidir cuándo y qué recuperar.

Mem0: memoria gestionada como servicio

Memo es un framework open source (y servicio cloud) que abstrae la capa de gestión de memoria para aplicaciones LLM. Su propuesta central es que la memoria debería ser un componente de infraestructura separado del modelo, al igual que la base de datos de una aplicación web es un componente separado del servidor de aplicaciones.

Memo gestiona automáticamente:

- La extracción de hechos memorables de cada conversación.
- La deduplicación y resolución de conflictos.

- El almacenamiento en una base de datos vectorial interna.
- La recuperación por similitud semántica.
- La actualización de memorias cuando la información cambia.

```
from mem0 import Memory

m = Memory()

# Añadir memorias desde una conversación
resultado = m.add(
    messages=[
        {"role": "user", "content": "Trabajo como analista de riesgo en un fondo de inv"},
        {"role": "assistant", "content": "Entendido, trabajaré con el contexto financiero"},
    ],
    user_id="laura_martinez"
)

# Recuperar memorias relevantes para una consulta
memorias = m.search(
    query="¿Qué tipo de informes prefiere este usuario?",
    user_id="laura_martinez"
)

for memoria in memorias:
    print(f"[{memoria['score']:.2f}] {memoria['memory']}")
```

Lo que Memo enseña al ingeniero: la gestión de memoria puede ser externalizada como servicio, con una API simple de add/search/update. Para proyectos que no justifican una implementación de memoria desde cero, este tipo de abstracción reduce enormemente la complejidad de desarrollo.

LangMem: integración de memoria en frameworks de agentes

LangMem es el sistema de memoria integrado en el ecosistema LangChain/LangGraph. Su diseño está orientado a agentes que mantienen conversaciones largas y necesitan memoria

persistente entre sesiones.

La arquitectura de LangMem distingue tres tipos de memoria con semánticas diferenciadas:

- **InMemoryStore:** almacenamiento en memoria del proceso, sin persistencia. Útil para estado temporal dentro de una sesión.
- **PostgresSaver / SqliteSaver:** almacenamiento de checkpoints de conversación, con persistencia y recuperación por `thread_id`.
- **Semantic memory stores:** integración con bases de datos vectoriales para recuperación por similitud.

Lo que distingue a LangMem es su integración nativa con el grafo de ejecución de agentes: la memoria no es un add-on sino un componente que el agente consulta y actualiza como parte de su ciclo de razonamiento.

El patrón de triple almacenamiento

Independientemente del framework usado, los sistemas de memoria de producción más maduros convergen en un patrón de triple almacenamiento:

TRIPLE STORAGE PATTERN		
KEY-VALUE	VECTOR STORE	RELATIONAL / DOCUMENT
Perfiles de usuario (acceso por ID exacto)	Memorias episódicas y semánticas (búsqueda por similitud)	Historial de conversaciones, logs de acciones, checkpoints
Redis DynamoDB	Qdrant/Pinecone Weaviate/Chroma	PostgreSQL/SQLite MongoDB

La ventaja de este patrón es que cada backend se usa para lo que hace mejor: key-value para acceso por ID exacto y alta velocidad, vectorial para recuperación semántica, relacional para queries estructurados y auditoría.

El costo es la complejidad operativa: tres sistemas de almacenamiento que mantener, sincronizar y respaldo. Este trade-off se justifica en sistemas de producción con alto volumen de usuarios; para sistemas pequeños o en desarrollo, una sola base de datos vectorial (que puede también manejar filtros por metadata) es frecuentemente suficiente.

Decisiones de diseño al elegir una arquitectura

La elección de arquitectura de memoria no debe hacerse en abstracto sino en función de los requisitos concretos del sistema:

Pregunta	Implicancia de diseño
¿Cuántos usuarios simultáneos habrá?	Escala del backend, uso de servicios cloud vs. self-hosted
¿Qué tan larga es la relación típica con el usuario?	Importancia de la memoria semántica vs. solo episódica
¿El sistema opera en tiempo real o puede tolerar latencia?	Recuperación sincrónica vs. asincrónica
¿Hay requisitos regulatorios sobre retención de datos?	Necesidad de TTL, eliminación garantizada, auditoría
¿El equipo tiene expertise en operación de bases de datos vectoriales?	Framework vs. implementación desde cero
¿El presupuesto de infraestructura es limitado?	Soluciones managed (Pinecone, Memo cloud) vs. self-hosted

No existe una arquitectura universalmente correcta. Existe la arquitectura que mejor resuelve los requisitos específicos del sistema dentro de las restricciones de equipo, tiempo y presupuesto.

La siguiente sección sistematiza los patrones de diseño que han demostrado funcionar bien en sistemas de memoria de producción: las soluciones repetibles para los problemas recurrentes que el ingeniero encontrará al construir este tipo de sistemas.

Capítulo 04 — Sección 09

Patrones de diseño

Un patrón de diseño es una solución repetible para un problema recurrente en un contexto dado. En el diseño de sistemas de memoria para IA, ciertos problemas aparecen una y otra vez independientemente del dominio de aplicación: cómo estructurar el perfil del usuario, cómo inyectar la memoria en el contexto sin saturarlo, cómo actualizar memoria sin perder información válida.

Esta sección presenta los patrones que han emergido como soluciones robustas a esos problemas. Para cada patrón: el problema que resuelve, la estructura de la solución y las condiciones en que aplica.

Patrón 1: Memory Store

Problema: el sistema necesita almacenar y recuperar información de usuario de forma confiable, pero la implementación directa mezcla lógica de negocio con lógica de almacenamiento.

Solución: encapsular todo el acceso a la memoria en un objeto o módulo dedicado con una interfaz clara: `guardar`, `recuperar`, `actualizar`, `eliminar`. El resto del sistema nunca accede directamente al backend de almacenamiento; solo interactúa con el Memory Store.


```

class MemoryStore:

    """Abstrae el acceso a la capa de almacenamiento de memoria."""

    def __init__(self, vector_store, kv_store):

        self._vector_store = vector_store

        self._kv_store = kv_store

    def guardar_hecho(self, user_id: str, hecho: str, tipo: str, metadata: dict = {}):

        """Guarda un hecho semántico sobre el usuario."""

        embedding = generar_embedding(hecho)

        self._vector_store.upsert(

            id=generar_id(user_id, hecho),

            embedding=embedding,

            metadata={"user_id": user_id, "tipo": tipo, "contenido": hecho, **metadata}

        )

    def recuperar_relevante(self, user_id: str, consulta: str, top_k: int = 5) -> list[

        """Recupera los hechos más relevantes para una consulta dada."""

        embedding_consulta = generar_embedding(consulta)

        return self._vector_store.buscar(

            embedding=embedding_consulta,

            filtros={"user_id": user_id},

            top_k=top_k

        )

    def obtener_perfil(self, user_id: str) -> dict:

        """Recupera el perfil estructurado del usuario."""

        return self._kv_store.get(f"perfil:{user_id}") or {}

    def eliminar_usuario(self, user_id: str):

        """Elimina toda la memoria de un usuario (para cumplimiento de privacidad)."""

        self._vector_store.eliminar(filtros={"user_id": user_id})

        self._kv_store.delete(f"perfil:{user_id}")

```

Cuándo aplica: siempre. Es el patrón base sobre el que construyen todos los demás.

Patrón 2: Context Assembler

Problema: la información disponible en la memoria (perfil, memorias episódicas, preferencias) supera el espacio disponible en el contexto. El sistema necesita decidir qué incluir y qué omitir para cada consulta específica.

Solución: un componente dedicado que recibe la consulta actual y ensambla el contexto óptimo para esa consulta, priorizando y filtrando la memoria disponible según relevancia y restricciones de tokens.

```
class ContextAssembler:

    def __init__(self, memory_store: MemoryStore, max_tokens_memoria: int = 1500):
        self._memory = memory_store

        self._max_tokens = max_tokens_memoria

    def ensamblar(self, user_id: str, consulta: str, historial: list[dict]) -> str:
        """
        Produce el bloque de contexto de memoria para inyectar en el prompt.
        Prioriza: perfil > memorias relevantes a la consulta > historial reciente.
        """
        perfil = self._memory.obtener_perfil(user_id)
        memorias_relevantes = self._memory.recuperar_relevante(
            user_id=user_id,
            consulta=consulta,
            top_k=8
        )

        # Construir bloque de contexto con control de tokens
        bloque = []
        tokens_usados = 0

        # Siempre incluir el perfil (alta prioridad)
        perfil_str = formatear_perfil(perfil)
        bloque.append(f"### Contexto del usuario\n{perfil_str}")
        tokens_usados += contar_tokens(perfil_str)

        # Añadir memorias relevantes hasta el límite de tokens
        memorias_str_list = []
        for memoria in memorias_relevantes:
            m_str = f"- {memoria['contenido']}"

            if tokens_usados + contar_tokens(m_str) > self._max_tokens:
                break

            memorias_str_list.append(m_str)
            tokens_usados += contar_tokens(m_str)
```

```
if memorias_str_list:
    bloque.append(f"### Memorias relevantes\n" + "\n".join(memorias_str_list))

return "\n\n".join(bloque)
```

Cuándo aplica: cuando el sistema tiene memoria persistente significativa y el riesgo de saturar el contexto con memoria es real.

Patrón 3: Memory Extractor

Problema: después de cada conversación, el sistema necesita identificar qué información nueva merece ser persistida, pero no es viable hacer eso manualmente ni guardar todo indiscriminadamente.

Solución: un paso dedicado de extracción que usa el LLM para identificar hechos memorables al final de la conversación (o de forma incremental durante la misma).

```
PROMPT_EXTRACCION = """Eres un sistema de extracción de memoria.

Analiza la conversación siguiente e identifica únicamente la información que
vale la pena recordar para futuras interacciones.

Criterios para incluir:

- Preferencias explícitas o implícitas del usuario
- Hechos sobre su rol, empresa, proyectos o dominio de trabajo
- Decisiones o acuerdos tomados que afectan trabajo futuro
- Restricciones o limitaciones mencionadas

Criterios para excluir:

- Preguntas exploratorias sin conclusión
- Contenido válido solo para esta sesión
- Información trivial o rutinaria

Responde en formato JSON con la lista de hechos a guardar:

{"hechos": [{"contenido": "...", "tipo": "preferencia|hecho|decision|restriccion", "con

Conversación:

{conversacion}"""

def extraer_memorias(conversacion: list[dict], llm) -> list[dict]:
    respuesta = llm.completar(
        prompt=PROMPT_EXTRACCION.format(conversacion=format_conversacion(conversacion)
    )
    return json.loads(respuesta) ["hechos"]
```

Cuándo aplica: cuando la captura de memoria es implícita (el sistema infiere qué guardar, no espera instrucciones explícitas del usuario).

Patrón 4: Memory Updater (Upsert semántico)

Problema: cuando el sistema intenta guardar un hecho nuevo, puede que ya exista un hecho similar o contradictorio en la memoria. Guardar ambos produce inconsistencias; descartar el nuevo puede perder información válida.

Solución: antes de insertar, buscar si existe un registro semánticamente similar. Si existe y el nuevo hecho lo contradice, actualizar. Si existe y el nuevo hecho lo confirma, reforzar su score de confianza. Si no existe, insertar.

```
def upsert_semantico(memory_store: MemoryStore, user_id: str, hecho_nuevo: dict):  
    """  
    Inserta o actualiza un hecho en memoria, resolviendo conflictos semánticamente.  
    """  
  
    similares = memory_store.recuperar_relevante(  
        user_id=user_id,  
        consulta=hecho_nuevo["contenido"],  
        top_k=3  
    )  
  
    for similar in similares:  
        if similar["score"] > 0.92: # Alta similitud: mismo hecho  
            if hay_contradiccion(similar["contenido"], hecho_nuevo["contenido"]):  
                # El nuevo hecho contradice al existente: actualizar  
                memory_store.actualizar(  
                    id=similar["id"],  
                    contenido=hecho_nuevo["contenido"],  
                    metadata={"actualizado_en": datetime.now().isoformat()}  
                )  
                return "actualizado"  
            else:  
                # El nuevo hecho confirma al existente: reforzar confianza  
                memory_store.incrementar_confianza(id=similar["id"])  
                return "confirmado"  
  
        # No hay similar: insertar como nuevo  
        memory_store.guardar_hecho(user_id, hecho_nuevo["contenido"], hecho_nuevo["tipo"])  
        return "insertado"
```

Cuándo aplica: en cualquier sistema que capture memoria implícita durante períodos extendidos. Es el patrón que previene la acumulación de memorias contradictorias.

Patrón 5: Sesión con Checkpoint

Problema: en sesiones largas o en agentes que operan durante horas, el estado de la sesión puede perderse por fallos del sistema, timeouts, o desconexiones del usuario.

Solución: guardar checkpoints del estado de la sesión (incluyendo el historial de conversación, el estado de las herramientas y cualquier output parcial) en almacenamiento persistente a intervalos regulares o en puntos críticos del flujo.

```
class SesionConCheckpoint:

    def __init__(self, session_id: str, storage):

        self.session_id = session_id

        self._storage = storage

        self.historial = []

        self.estado = {}

        self._cargar_checkpoint()

    def _cargar_checkpoint(self):

        checkpoint = self._storage.get(f"checkpoint:{self.session_id}")

        if checkpoint:

            self.historial = checkpoint["historial"]

            self.estado = checkpoint["estado"]

    def guardar_checkpoint(self):

        self._storage.set(

            key=f"checkpoint:{self.session_id}",

            value={"historial": self.historial, "estado": self.estado},

            ttl=86400 # 24 horas

        )

    def agregar_turno(self, role: str, content: str):

        self.historial.append({"role": role, "content": content})

        if len(self.historial) % 5 == 0: # Checkpoint cada 5 turnos

            self.guardar_checkpoint()
```

Cuándo aplica: en agentes autónomos de larga duración, en aplicaciones donde la sesión puede interrumpirse, en cualquier contexto donde perder el estado de sesión tiene costo significativo para el usuario.

La siguiente sección examina los anti-patrones: los errores de diseño más comunes en sistemas de memoria de IA y por qué producen los comportamientos problemáticos que producen.

Capítulo 04 — Sección 10

Anti-patrones

Los anti-patrones son soluciones que parecen razonables —o que son consecuencia natural de tomar el camino de menor resistencia— pero que producen problemas predecibles en producción. Esta sección describe los anti-patrones más frecuentes en sistemas de memoria de IA, sus síntomas y las correcciones que los resuelven.

Anti-patrón 1: La memoria esponja (guardar todo)

Descripción: el sistema guarda cada mensaje, cada dato mencionado en cada conversación, sin ningún criterio de selectividad. La premisa implícita es "más memoria es mejor".

Por qué ocurre: es la implementación de menor esfuerzo. Si no hay criterio de qué guardar, guardar todo evita el problema de decidir.

Síntomas en producción:

- La base de datos de memoria crece sin control.
- La recuperación semántica devuelve memorias irrelevantes que contaminan el contexto.
- El sistema recuerda detalles triviales ("el usuario preguntó qué temperatura hace en Madrid") con la misma prominencia que información crítica.
- Las respuestas del modelo se vuelven ruidosas porque el contexto inyectado contiene memorias irrelevantes mezcladas con las relevantes.
- El costo operativo (almacenamiento + computación de embeddings) crece de forma no sostenible.

Corrección: implementar un componente de captura selectiva (Memory Extractor, patrón de la sección anterior) con criterios explícitos de qué merece ser persistido. El criterio mínimo: ¿esta información tiene valor en una sesión futura? Si la respuesta no es claramente sí, no persiste.

Anti-patrón 2: El contexto desbordado (memory dumping)

Descripción: el sistema recupera toda la memoria disponible sobre el usuario y la inyecta en el contexto al inicio de cada sesión, sin importar su relevancia para la consulta actual.

Por qué ocurre: es el equivalente de "mejor que sobre que no falte". El sistema no tiene un componente de Context Assembly, así que vuelca todo lo que encuentra.

Síntomas en producción:

- El costo de tokens por sesión es alto incluso para consultas simples.
- Las respuestas del modelo muestran referencias a memorias irrelevantes ("ya sé que preferís Python, aunque tu pregunta sea sobre una hoja de cálculo").
- El fenómeno "lost in the middle": el modelo pierde el foco en la consulta real porque está enterrada bajo demasiado contexto de memoria.
- En conversaciones largas, se alcanza el límite de contexto prematuramente porque la memoria inyectada ocupa demasiado espacio.

Corrección: implementar un Context Assembler que seleccione y priorice la memoria según su relevancia para la consulta actual. La memoria inyectada debe ser suficiente para contextualizar, no exhaustiva.

Anti-patrón 3: La memoria muerta (nunca se actualiza)

Descripción: el sistema guarda información la primera vez que la captura pero nunca la actualiza, incluso cuando el usuario provee información más reciente o contradictoria.

Por qué ocurre: el sistema tiene mecanismo de escritura pero no de actualización. Cada hecho nuevo se guarda como registro nuevo, sin verificar si existe un registro anterior sobre el mismo tema.

Síntomas en producción:

- La memoria contiene versiones contradictorias del mismo hecho: "el usuario trabaja en empresa A" y "el usuario trabaja en empresa B".
- El modelo recibe ambas versiones en el contexto y produce respuestas ambiguas o elige arbitrariamente una de las dos.
- El usuario corrige al sistema ("te dije que ahora trabajo en X") pero el sistema sigue usando la información anterior en sesiones futuras.
- La confianza del usuario en el sistema se degrada.

Corrección: implementar el patrón Upsert Semántico (sección 09). Antes de guardar cualquier hecho nuevo, verificar si existe un registro con alta similitud semántica. Si existe, actualizar en lugar de insertar.

Anti-patrón 4: La memoria fantasma (sin TTL ni expiración)

Descripción: toda la memoria persiste indefinidamente. No hay mecanismo de expiración, archivado ni eliminación por caducidad.

Por qué ocurre: el equipo se concentró en el diseño de captura y recuperación, y no pensó en el ciclo de vida de la memoria.

Síntomas en producción:

- El sistema usa información desactualizada con plena confianza: menciona proyectos que cerraron, habla de restricciones que ya no aplican, asume un contexto organizacional que cambió.
- Los usuarios experimentan que el sistema "está viviendo en el pasado".
- A medida que pasa el tiempo, la proporción de memorias desactualizadas crece, degradando la calidad general del sistema.

Corrección: implementar TTL diferenciado por tipo de información (como se describió en la sección 07). Los hechos volátiles expiran rápido; los hechos estables expiran lentamente o cuando son explícitamente contradichos.

Anti-patrón 5: La memoria opaca (sin visibilidad para el usuario)

Descripción: el sistema tiene memoria persistente pero el usuario no sabe qué recuerda sobre él, no puede consultarla y no puede corregirla.

Por qué ocurre: el equipo implementó la memoria como un componente interno sin interfaz de usuario. La transparencia no se consideró un requisito de diseño.

Síntomas en producción:

- El usuario percibe que el sistema "inventó" preferencias que nunca expresó.
- El usuario no puede corregir una memoria incorrecta excepto repitiendo la corrección hasta

que el sistema la captura.

- El usuario no sabe si sus datos están siendo guardados, qué datos son o durante cuánto tiempo.
- En contextos empresariales, esto puede generar desconfianza o directamente violar regulaciones de privacidad.

Corrección: diseñar una interfaz de gestión de memoria —aunque sea mínima— que permita al usuario ver qué recuerda el sistema sobre él y eliminar o corregir entradas específicas. Esta interfaz puede ser tan simple como un comando `/memoria` dentro del chat, o tan elaborada como una pantalla de configuración de perfil.

Anti-patrón 6: La memoria centralizada en el prompt de sistema (hardcoding de contexto)

Descripción: en lugar de diseñar memoria dinámica, el equipo escribe la información del usuario directamente en el system prompt de forma estática. El "diseño de memoria" consiste en un texto fijo que dice "eres un asistente para el equipo de finanzas de empresa X".

Por qué ocurre: es la solución de prototipo que escala mal. Funciona para un sistema de un solo cliente y un solo contexto, pero no escala a múltiples usuarios con contextos distintos.

Síntomas en producción:

- El sistema funciona bien para el caso de uso para el que fue configurado y falla para cualquier variación.
- Todos los usuarios reciben exactamente el mismo contexto, sin personalización.
- Actualizar el contexto requiere modificar y redespigar el system prompt.
- Es imposible adaptar el sistema a nuevos usuarios sin intervención de ingeniería.

Corrección: separar el system prompt estático (instrucciones de comportamiento) de la memoria dinámica (contexto del usuario). El system prompt define cómo debe comportarse el sistema; la memoria dinámica provee el contexto específico de cada usuario, recuperado en tiempo de ejecución.

Tabla resumen de anti-patrones

Anti-patrón	Síntoma principal	Corrección
Memoria esponja	Ruido en recuperación, costo descontrolado	Captura selectiva con criterios explícitos
Context dumping	Context saturado, respuestas ruidosas	Context Assembler por relevancia
Memoria muerta	Contradicciones, información desactualizada	Upsert semántico con detección de conflictos
Memoria fantasma	"Viviendo en el pasado"	TTL diferenciado por tipo de dato
Memoria opaca	Desconfianza, incapacidad de corregir	Interfaz de gestión de memoria para el usuario
Hardcoding de contexto	Sin personalización, no escala	Memoria dinámica separada del system prompt

La siguiente sección presenta un caso de estudio empresarial concreto que integra los patrones vistos y muestra las decisiones de diseño tomadas en un sistema real de producción.

Capítulo 04 — Sección 11

Caso de estudio empresarial

Sistema de asistencia al equipo de consultoría — Firma de servicios profesionales

Esta sección traza la evolución de un sistema de asistencia de IA para un equipo de consultores de gestión. El caso es representativo de los problemas que cualquier equipo enfrenta cuando pasa de un prototipo de chat a un sistema de producción con múltiples usuarios, sesiones largas y expectativa de continuidad.

Los detalles específicos de la empresa son compuestos y anonimizados, pero los problemas y las soluciones son directamente derivados de patrones observados en implementaciones reales.

El contexto

Una firma de consultoría de gestión con 80 consultores implementó un asistente de IA interno para apoyar el trabajo de investigación, redacción de propuestas y análisis de datos de sus proyectos. El piloto inicial fue un chatbot simple basado en un LLM con acceso a la documentación interna de la empresa.

El piloto fue bien recibido. Los consultores lo usaban para buscar metodologías de proyectos anteriores, redactar borradores de secciones de propuestas y explorar benchmarks de industria. Sin embargo, a los tres meses del piloto, emergieron quejas recurrentes:

- "Tengo que explicarle quién soy cada vez que abro el chat."
- "Sabe que estoy trabajando en el proyecto Antares, pero en la próxima sesión no recuerda nada."
- "Me propuso una metodología que yo mismo le dije que no aplica en nuestro contexto hace dos semanas."

El equipo de tecnología diagnosticó el problema correctamente: el sistema no tenía memoria persistente. Cada sesión comenzaba desde cero.

La decisión de diseño: qué recordar

El primer debate fue sobre qué debería recordar el sistema. El equipo identificó tres categorías de información con distintos valores y distintos requisitos de actualización:

Perfil del consultor (memoria semántica estable):

- Nombre, rol y nivel de seniority
- Áreas de especialización (industrias, funciones de negocio, geografías)
- Proyectos activos y rol en cada uno
- Metodologías y frameworks preferidos
- Formato de trabajo preferido (más o menos estructura en los outputs)

Contexto de proyecto (memoria episódica con TTL por proyecto):

- Nombre del cliente y sector
- Objetivo central del proyecto y entregables
- Hipótesis de trabajo actuales
- Restricciones y limitaciones conocidas (confidencialidad, acceso a datos, presupuesto)
- Decisiones tomadas en iteraciones anteriores

Historial de sesión (memoria conversacional con ventana deslizante):

- Los últimos 8 turnos de la conversación actual
- Resumen comprimido de los turnos anteriores de la misma sesión

La decisión explícita fue no guardar: el contenido de los documentos del cliente (por confidencialidad), las consultas de exploración sin conclusión, y cualquier información que el consultor no hubiera validado (para evitar guardar inferencias incorrectas).

La arquitectura implementada

El equipo eligió una arquitectura de doble backend:

- **Redis** para los perfiles de consultor (acceso por ID, baja latencia requerida).
- **Qdrant** (self-hosted, por requerimiento de privacidad) para las memorias de proyecto y las memorias episódicas, con recuperación semántica.

La capa de aplicación implementó los patrones de Memory Store, Context Assembler y Memory Extractor de las secciones anteriores.

FLUJO POR SESIÓN:

1. Usuario inicia sesión → se carga perfil del consultor desde Redis
2. Usuario selecciona proyecto activo → se recuperan memorias del proyecto desde Qdrant
3. Context Assembler construye el bloque de contexto:
[perfil del consultor] + [memorias del proyecto relevantes] + [historial de sesión]
4. Durante la sesión: ventana deslizante + resumen progresivo
5. Al cerrar sesión: Memory Extractor identifica hechos memorables
6. Upsert semántico: los hechos nuevos se integran sin crear duplicados

El bloque de contexto de memoria tenía un límite de 1.200 tokens —un compromiso entre cobertura y costo.

Los problemas que surgieron en producción

Problema 1: El consultor se va del proyecto pero la memoria no

A los dos meses del despliegue, un consultor que había rotado fuera del proyecto Antares recibía memorias del proyecto Antares en sus sesiones de un proyecto completamente distinto. El Qdrant devolvía memorias del proyecto anterior porque tenían alta similitud semántica con las consultas nuevas (ambos proyectos eran del sector retail).

Solución: añadir un filtro de `proyecto_activo` en la recuperación. Las memorias de proyecto solo se recuperan cuando el proyecto está marcado como activo para ese consultor. Al cerrar un proyecto, sus memorias se archivan (se marca el campo `activo: false`) y quedan fuera de los resultados de recuperación estándar.

Problema 2: El resumen de sesión perdía detalles críticos

El resumen progresivo de sesiones largas perdía a veces restricciones importantes que el consultor había mencionado temprano en la sesión ("no podemos proponer soluciones que impliquen cambios en el ERP"). El prompt de resumen no las priorizaba adecuadamente.

Solución: modificar el prompt de resumen para que identifique y preserve explícitamente las restricciones y los acuerdos antes de comprimir el contenido narrativo. Se añadió un bloque dedicado de `restricciones_de_sesión` en el contexto, con alta prioridad de preservación.

Problema 3: Un consultor rechazó la memoria, generando un incidente de privacidad

Un consultor senior descubrió que el sistema había inferido y guardado como "preferencia" algo que él consideraba una hipótesis de trabajo, no una decisión permanente. Solicitó saber exactamente qué tenía el sistema guardado sobre él y quería eliminarlo todo.

Solución: el equipo implementó en una semana un endpoint de API que devolvía todos los registros de memoria asociados a un `user_id`. Dentro del chat, el comando `/mi-memoria` listaba todas las memorias del perfil. El comando `/olvidar [texto]` permitía eliminar registros específicos. La eliminación completa de un usuario fue probada y documentada como procedimiento.

Este incidente llevó al equipo a revisar el criterio de captura de memorias inferidas: solo se guardan con el tipo `inferida` y con un score de confianza `media`, diferenciado de los hechos `explicitos` que el consultor expresó directamente.

Resultados a los seis meses

Después de seis meses de operación con la arquitectura de memoria:

- El 78% de los consultores reportó que el sistema "los conoce" y produce respuestas más relevantes que en el piloto inicial.
- El promedio de turnos por sesión para llegar a un output útil bajó de 6,2 a 3,8, atribuido a la reducción de contexto explicativo que el consultor necesita dar al inicio de cada sesión.
- El equipo de tecnología identificó que el 23% de las memorias capturadas era ruido (información sin valor para sesiones futuras). La revisión del criterio de extracción mejoró este indicador.
- El primer incidente de privacidad resultó en la implementación de controles que el equipo reconoció como necesarios desde el inicio pero que habían postergado.

Lecciones del caso

La memoria no es un feature, es una promesa. Cuando el sistema empieza a recordar a un usuario, ese usuario desarrolla expectativas de continuidad. Si esas expectativas se rompen —porque la memoria falla, porque guarda mal, porque no actualiza— la confianza se degrada más rápido que si el sistema nunca hubiera tenido memoria.

La privacidad del usuario es un requisito de diseño, no un requisito de cumplimiento. El equipo lo aprendió por un incidente. Un diseño proactivo habría evitado el incidente.

Los bugs de memoria son difíciles de diagnosticar. Cuando el sistema produce una respuesta subóptima por culpa de una memoria incorrecta, el usuario generalmente no lo sabe: atribuye el error al modelo, no a la memoria. Esto hace que los bugs de memoria sean silenciosos y su impacto, subestimado.

La siguiente sección es el laboratorio práctico: el estudiante implementa un sistema de memoria persistente simple —basado en JSON—, con criterios explícitos de captura, recuperación y eliminación.

Capítulo 04 — Sección 12

Laboratorio práctico

Objetivo del laboratorio

Implementar un sistema de memoria persistente funcional usando JSON como backend de almacenamiento. El sistema debe ser capaz de: capturar información relevante de una conversación, persistirla entre sesiones, recuperar memorias relevantes para una consulta nueva, y eliminar la memoria de un usuario.

Al finalizar este laboratorio, habrás construido el esqueleto de un sistema de memoria que puede ser extendido con un backend más robusto (Redis, Qdrant) sin cambiar la lógica de negocio.

Prerrequisitos

- Python 3.10 o superior
- Un cliente de API de LLM con acceso a un modelo que soporte function calling (Claude, GPT-4, etc.)
- Instalación: `pip install anthropic` (o el cliente de tu proveedor de API)

Estructura del proyecto

```
memoria_lab/  
├─ memoria.py          # Motor de memoria  
├─ extractor.py        # Extracción de hechos memorables  
├─ asistente.py        # Integración con el LLM  
├─ datos/  
│   └─ memorias.json   # Base de datos de memoria (creada automáticamente)  
└─ main.py             # Punto de entrada
```

Paso 1: El motor de memoria (memoria.py)

```
import json
import os

from datetime import datetime
from pathlib import Path

RUTA_MEMORIAS = Path("datos/memorias.json")

def cargar_memorias() -> dict:
    """Carga la base de datos de memoria desde disco."""
    if not RUTA_MEMORIAS.exists():
        RUTA_MEMORIAS.parent.mkdir(exist_ok=True)
        return {}
    with open(RUTA_MEMORIAS, "r", encoding="utf-8") as f:
        return json.load(f)

def guardar_memorias(memorias: dict):
    """Persiste la base de datos de memoria en disco."""
    with open(RUTA_MEMORIAS, "w", encoding="utf-8") as f:
        json.dump(memorias, f, ensure_ascii=False, indent=2)

def guardar_hecho(user_id: str, hecho: str, tipo: str):
    """
    Guarda un hecho sobre el usuario.
    Verifica duplicados antes de insertar.
    """
    memorias = cargar_memorias()

    if user_id not in memorias:
        memorias[user_id] = {"hechos": [], "perfil": {}}

    # Verificar duplicado simple (comparación de texto)
    existentes = [h["contenido"] for h in memorias[user_id]["hechos"]]
    if hecho not in existentes:
        memorias[user_id]["hechos"].append({
            "id": f"{user_id}_{len(memorias[user_id]['hechos'])}",

```



```
        "contenido": hecho,

        "tipo": tipo,

        "creado": datetime.now().isoformat(),

        "ultimo_uso": datetime.now().isoformat()

    })

    guardar_memorias(memorias)

    return True

return False # Ya existía

def recuperar_memorias(user_id: str) -> list[dict]:

    """

    Recupera todos los hechos de un usuario.

    (En producción: recuperación semántica por similitud)

    """

    memorias = cargar_memorias()

    if user_id not in memorias:

        return []

    return memorias[user_id]["hechos"]

def eliminar_memoria_usuario(user_id: str) -> int:

    """

    Elimina toda la memoria de un usuario.

    Retorna el número de registros eliminados.

    """

    memorias = cargar_memorias()

    if user_id not in memorias:

        return 0

    count = len(memorias[user_id]["hechos"])

    del memorias[user_id]

    guardar_memorias(memorias)

    return count

def eliminar_hecho(user_id: str, hecho_id: str) -> bool:

    """Elimina un hecho específico por ID."""

    memorias = cargar_memorias()
```

```
if user_id not in memorias:
    return False

antes = len(memorias[user_id]["hechos"])

memorias[user_id]["hechos"] = [
    h for h in memorias[user_id]["hechos"] if h["id"] != hecho_id
]

if len(memorias[user_id]["hechos"]) < antes:
    guardar_memorias(memorias)

    return True

return False
```

Paso 2: El extractor de memorias (extractor.py)

```

import json

PROMPT_EXTRACCION = """Analiza la conversación y extrae los hechos que vale la pena recordar.

Solo extrae:

- Preferencias explícitas del usuario (formato, herramientas, idioma, etc.)
- Hechos sobre su trabajo, empresa o proyectos
- Decisiones o acuerdos relevantes para trabajo futuro
- Restricciones o limitaciones expresadas

NO extraigas:

- Preguntas exploratorias sin conclusión
- Contenido relevante solo para esta sesión
- Saludos o frases rutinarias

Responde SOLO con JSON válido, sin texto adicional:

{{
  "hechos": [
    {{ "contenido": "descripción del hecho", "tipo": "preferencia|trabajo|decision|restriccion" }}
  ]
}}

Si no hay nada que valga la pena recordar, responde: {{ "hechos": [] }}

Conversación:
{conversacion}"""

def extraer_hechos_memorables(conversacion: list[dict], llm_client) -> list[dict]:
    """
    Usa el LLM para identificar qué información de la conversación
    merece ser persistida en memoria.
    """
    if not conversacion:
        return []

```

```
texto_conversacion = "\n".join([
    f"{'Usuario' if m['role'] == 'user' else 'Asistente'}: {m['content']}"
    for m in conversacion
])

respuesta = llm_client.completar(
    prompt=PROMPT_EXTRACCION.format(conversacion=texto_conversacion)
)

try:
    datos = json.loads(respuesta)
    return datos.get("hechos", [])
except json.JSONDecodeError:
    return []
```

Paso 3: El asistente con memoria (asistente.py)

```

from memoria import recuperar_memorias, guardar_hecho
from extractor import extraer_hechos_memorables

SYSTEM_PROMPT_BASE = """Eres un asistente de análisis. Ayudas al usuario con su trabajo

{bloque_memoria}

Usa el contexto de memoria para dar respuestas personalizadas y relevantes.
Si la memoria está vacía, trata la sesión como la primera vez que hablas con este usuario

def construir_bloque_memoria(user_id: str) -> str:
    """Construye el bloque de contexto de memoria para inyectar en el prompt."""
    hechos = recuperar_memorias(user_id)

    if not hechos:
        return """## Memoria del usuario\n(Sin memorias previas)"""

    lineas = ["## Memoria del usuario"]
    for hecho in hechos[-10:]: # Máximo 10 hechos para este laboratorio
        lineas.append(f"- [{hecho['tipo']}] {hecho['contenido']}")

    return "\n".join(lineas)

class AsistenteConMemoria:
    def __init__(self, user_id: str, llm_client):
        self.user_id = user_id
        self.llm = llm_client
        self.historial = []

    def responder(self, mensaje_usuario: str) -> str:
        """Procesa un mensaje del usuario y produce una respuesta."""
        # Construir el contexto con memoria
        bloque_memoria = construir_bloque_memoria(self.user_id)
        system_prompt = SYSTEM_PROMPT_BASE.format(bloque_memoria=bloque_memoria)

```

```
# Agregar mensaje al historial
self.historial.append({"role": "user", "content": mensaje_usuario})

# Llamar al LLM
respuesta = self.llm.chat(
    system=system_prompt,
    messages=self.historial
)

# Agregar respuesta al historial
self.historial.append({"role": "assistant", "content": respuesta})

return respuesta

def cerrar_sesion(self):
    """
    Al finalizar la sesión, extrae y persiste los hechos memorables.
    """
    print("\n[Sistema] Guardando memorias de la sesión...")
    hechos = extraer_hechos_memorables(self.historial, self.llm)

    guardados = 0
    for hecho in hechos:
        exito = guardar_hecho(
            user_id=self.user_id,
            hecho=hecho["contenido"],
            tipo=hecho["tipo"]
        )
        if exito:
            guardados += 1
            print(f"  + Guardado: {hecho['contenido']}")

    print(f"[Sistema] {guardados} memorias nuevas guardadas.")
```

Paso 4: Punto de entrada (main.py)

```
from asistente import AsistenteConMemoria

from memoria import recuperar_memorias, eliminar_memoria_usuario

# Importar tu cliente de LLM aquí


USER_ID = "usuario_demo"


def main():

    # Inicializar el cliente LLM (reemplazar con tu implementación)

    llm = TuClienteLLM()


    asistente = AsistenteConMemoria(user_id=USER_ID, llm_client=llm)


    print("=== Asistente con Memoria ===")
    print("Comandos especiales: /memoria, /olvidar-todo, /salir\n")


    while True:

        entrada = input("Tú: ").strip()


        if not entrada:

            continue


        # Comandos especiales

        if entrada == "/salir":

            asistente.cerrar_sesion()

            print("Hasta luego.")

            break


        elif entrada == "/memoria":

            hechos = recuperar_memorias(USER_ID)

            if not hechos:

                print("[Memoria vacía]")

            else:

                print(f"[{len(hechos)} hechos guardados sobre ti:]")

                for h in hechos:

                    print(f"    [{h['id']}] ({h['tipo']}) {h['contenido']}")
```

```

        continue

    elif entrada == "/olvidar-todo":
        confirmacion = input("¿Seguro? Esto eliminará toda tu memoria. (sí/no): ")

        if confirmacion.lower() == "sí":
            eliminados = eliminar_memoria_usuario(USER_ID)
            print(f"[{eliminados} registros eliminados]")

            continue

        # Respuesta normal
        respuesta = asistente.responder(entrada)
        print(f"Asistente: {respuesta}\n")

if __name__ == "__main__":
    main()

```

Ejercicios de extensión

Una vez que el sistema base funciona, los siguientes ejercicios aumentan su sofisticación:

Ejercicio 1 — TTL automático: modificar `guardar_hecho` para aceptar un parámetro `ttl_dias`. Modificar `recuperar_memorias` para filtrar automáticamente los hechos cuya fecha de creación supera el TTL.

Ejercicio 2 — Recuperación semántica: reemplazar la recuperación simple (todos los hechos del usuario) por recuperación semántica usando la API de embeddings de tu proveedor y búsqueda por similitud coseno. Implementar la función `recuperar_hechos_relevantes(user_id, consulta, top_k)`.

Ejercicio 3 — Detección de contradicciones: modificar `guardar_hecho` para detectar contradicciones usando el LLM. Si el hecho nuevo contradice a uno existente, actualizar el existente en lugar de insertar uno nuevo.

Ejercicio 4 — Migración a Qdrant: reemplazar el backend JSON por Qdrant (instalable con `pip install qdrant-client`). La interfaz de `memoria.py` debería permanecer igual — solo cambia la implementación interna.

La siguiente sección presenta el checklist del AI Engineer: las preguntas que todo diseñador de sistemas con memoria debería responder antes de ir a producción.

Capítulo 04 — Sección 13

Checklist del AI Engineer

Este checklist cubre las decisiones de diseño que deben estar respondidas antes de llevar un sistema con memoria a producción. No es una lista de verificación de implementación — asume que la implementación ya está hecha— sino una lista de preguntas de diseño que, si no tienen respuesta, indican áreas de riesgo.

Organiza el checklist en cinco dimensiones: captura, almacenamiento, recuperación, ciclo de vida y privacidad.

Dimensión 1: Captura

¿Tengo criterios explícitos de qué se guarda y qué no?

No basta con "lo que parece importante". Los criterios deben estar documentados y deben ser verificables: dado un fragmento de conversación, cualquier miembro del equipo debería poder decir si ese fragmento se guarda o no según los criterios.

¿El sistema distingue entre hechos explícitos e inferidos?

Los hechos que el usuario dijo directamente tienen mayor confiabilidad que los que el sistema infirió. El diseño debe reflejar esta diferencia —en el almacenamiento, en la recuperación y en cómo se presentan al modelo.

¿El usuario sabe que sus datos están siendo capturados?

No necesariamente con un pop-up legal, pero sí hay una expectativa razonable de transparencia. ¿La documentación del sistema lo menciona? ¿Hay alguna señal en la interfaz?

¿El sistema captura en tiempo real o al cierre de sesión?

Ambas opciones son válidas, pero la elección tiene consecuencias: la captura en tiempo real puede guardar fragmentos incompletos; la captura al cierre puede perder información si la sesión se interrumpe abruptamente.

Dimensión 2: Almacenamiento

¿El backend elegido es el correcto para el patrón de recuperación?

Recuperación por ID exacto → key-value. Recuperación por similitud semántica → base de datos vectorial. Recuperación con queries complejas o joins → relacional. ¿El backend que elegiste hace bien lo que necesitas?

¿Toda la memoria tiene `user_id` como metadato indexado?

Es el requisito mínimo para poder: recuperar toda la memoria de un usuario, eliminar toda la memoria de un usuario, aislar la memoria entre usuarios. Sin `user_id` indexado, estas operaciones son caras o imposibles.

¿El volumen de memoria esperado está dentro de los límites del backend elegido?

Una base de datos vectorial self-hosted en un servidor modesto puede manejar millones de vectores. Una base de datos vectorial en memoria (Chroma sin persistencia) no escala más allá de decenas de miles. ¿Hiciste la estimación de volumen?

¿Hay backups y recuperación ante fallos para el almacenamiento de memoria?

La memoria es datos de usuario. Perder la base de datos de memoria no es solo un problema técnico —es una degradación de servicio que el usuario notará inmediatamente.

Dimensión 3: Recuperación

¿La recuperación filtra correctamente por usuario?

En una base de datos vectorial sin filtro por `user_id`, una búsqueda semántica puede devolver memorias de otros usuarios con alta similitud. Este es un bug de privacidad, no solo un bug de calidad.

¿Cuántos tokens ocupa la memoria inyectada en el peor caso?

Si tienes un usuario con 500 memorias y no hay límite de recuperación, una consulta puede inyectar miles de tokens en el contexto. ¿Hay un límite definido de tokens o registros para la inyección de memoria?

¿La recuperación tiene latencia aceptable?

Una búsqueda vectorial sobre una colección pequeña toma milisegundos. Sobre una colección grande, puede tomar segundos. ¿El sistema tiene un timeout de recuperación? ¿Qué sucede si la recuperación falla?

¿La memoria recuperada es relevante para el tipo de consulta?

Haz una prueba manual: crea un conjunto de memorias de prueba para un usuario y verifica que para diez consultas distintas, la memoria recuperada es la que efectivamente ayuda a responder.

Dimensión 4: Ciclo de vida

¿Hay políticas de TTL o expiración para distintos tipos de memoria?

Si toda la memoria persiste indefinidamente, el sistema producirá comportamientos basados en información desactualizada. ¿Tienes TTL diferenciados por tipo de dato?

¿El sistema resuelve conflictos cuando captura información contradictoria?

Sin resolución de conflictos, la memoria acumula versiones contradictorias del mismo hecho. ¿El sistema detecta y resuelve esto?

¿Hay un proceso de consolidación para memorias episódicas acumuladas?

Con el tiempo, docenas de memorias episódicas sobre el mismo tema deberían consolidarse en una memoria semántica. Sin consolidación, el sistema crece sin control y la recuperación se degrada.

¿El sistema ha sido probado después de 6 meses de uso simulado?

Muchos problemas de memoria —acumulación de ruido, información desactualizada, conflictos no resueltos— solo emergen después de un período prolongado de uso. ¿Tienes un test de carga temporal?

Dimensión 5: Privacidad y control del usuario

¿El usuario puede ver qué recuerda el sistema sobre él?

Si la respuesta es no, el usuario no tiene ningún mecanismo de control sobre su propia información. Esto es un problema de diseño y potencialmente un problema legal.

¿El usuario puede eliminar memorias específicas?

Ver no es suficiente. El usuario debe poder eliminar registros incorrectos o que no quiere que el sistema recuerde.

¿La eliminación completa de memoria de un usuario está implementada y probada?

No solo "implementada" en el sentido de que hay una función. Probada: ejecuta la eliminación para un usuario de prueba y verifica que no queda ningún registro —ni en el vector store, ni en el key-value, ni en ningún caché.

¿El sistema cumple con las regulaciones de privacidad aplicables en los países donde opera?

GDPR (Europa), LGPD (Brasil), Ley 25.326 (Argentina), CCPA (California) tienen distintos requisitos sobre retención, eliminación y transparencia. ¿El equipo legal revisó el diseño?

¿Hay un proceso documentado para responder solicitudes de eliminación?

Cuando un usuario solicita que se eliminen sus datos, ¿hay un proceso claro de quién lo recibe, quién lo ejecuta y cómo se confirma que se ejecutó correctamente?

Puntuación del checklist

Cada pregunta del checklist tiene tres posibles estados:

- **Sí / Resuelto:** el diseño contempla esto explícitamente.
- **Parcialmente / En progreso:** existe una solución incompleta o en desarrollo.
- **No / Pendiente:** no hay diseño para esto todavía.

Un sistema listo para producción debería tener todas las preguntas en "Sí" o con un plan documentado para las que están en "Parcialmente". Cualquier pregunta en "No" representa un riesgo conocido que el equipo debería aceptar de forma explícita, no ignorar.

La siguiente sección es el resumen del capítulo: los conceptos centrales consolidados, las conexiones con otros capítulos del módulo, y las ideas que el lector debería llevarse como puntos de partida para su propio trabajo de diseño.

Capítulo 04 — Sección 14

Resumen del capítulo

Este capítulo convirtió la memoria de una estrategia de administración del contexto —como la nombramos en el capítulo anterior— en una disciplina de diseño con componentes, patrones y criterios propios.

Los conceptos centrales

La taxonomía cognitiva como marco de diseño. Los cuatro tipos de memoria de la psicología cognitiva —de trabajo, episódica, semántica y procedimental— mapean directamente sobre los componentes de un sistema de memoria de IA. Este mapeo nos dio el vocabulario para distinguir tipos de información que requieren tratamientos radicalmente distintos: la ventana de contexto es memoria de trabajo; el historial de conversaciones es memoria episódica; el perfil del usuario es memoria semántica; las instrucciones del sistema son memoria procedimental.

Los cinco componentes de la arquitectura de memoria. Todo sistema de memoria pasa por los mismos cinco pasos: captura (¿qué vale la pena recordar?), procesamiento (¿cómo se estructura?), almacenamiento (¿dónde y con qué backend?), recuperación (¿qué es relevante para esta consulta?) e inyección (¿cómo se incorpora al contexto activo?). Un fallo en cualquiera de estos componentes degrada la calidad de todo el sistema.

Las estrategias de memoria conversacional. Para gestionar el historial dentro de una sesión, hay cuatro enfoques con trade-offs distintos: historial completo (máxima coherencia, costo alto), ventana deslizante (costo controlado, pérdida de contexto antiguo), resumen progresivo (preserva información a menor costo en tokens) e híbrida (mayor robustez, mayor complejidad). La elección depende de la longitud típica de las conversaciones y de la criticidad de la información histórica.

Los backends de almacenamiento persistente y cuándo elegir cada uno. Key-value stores para perfiles estructurados de acceso por ID. Bases de datos vectoriales para recuperación semántica por similitud. Grafos de conocimiento para relaciones complejas entre entidades. La elección del backend debe seguir al patrón de recuperación, no al revés.

La distinción entre memoria semántica y RAG. La memoria semántica es conocimiento construido por la aplicación sobre el usuario y el dominio de uso. RAG es conocimiento recuperado de documentos externos. Ambos usan tecnologías similares, pero resuelven problemas distintos. Un sistema completo puede tener ambos; un ingeniero que los confunde diseña mal ambos.

El olvido como función de diseño. La consolidación y el olvido deliberado son componentes tan críticos como la captura. Los sistemas que no diseñan el olvido acumulan ruido, mantienen información desactualizada y crecen sin control. Las políticas de retención deben ser explícitas, con TTL diferenciados por tipo de información y mecanismos de resolución de conflictos cuando la nueva información contradice la anterior.

Los patrones que funcionan. Memory Store (abstracción del backend), Context Assembler (selección y priorización para inyección), Memory Extractor (captura selectiva vía LLM), Memory Updater con upsert semántico (resolución de conflictos), y Sesión con Checkpoint (resiliencia ante interrupciones).

Los anti-patrones que no funcionan. Memoria esponja (guardar todo), context dumping (inyectar todo sin filtro), memoria muerta (sin actualización), memoria fantasma (sin TTL), memoria opaca (sin control del usuario) y hardcoding de contexto (contexto estático en lugar de memoria dinámica).

Las conexiones con el resto del módulo

Este capítulo es el punto de articulación entre varios temas del módulo:

Hacia atrás: la memoria es la cuarta estrategia de administración del contexto que el capítulo 03 identificó. Este capítulo la desarrolló en profundidad técnica.

Hacia adelante — Capítulo 06 (RAG): la memoria semántica y RAG son complementarios. Este capítulo estableció que la memoria semántica gestiona el conocimiento sobre el usuario y el dominio de uso; RAG gestiona el conocimiento sobre documentos externos. El capítulo 06 desarrollará la arquitectura RAG en su totalidad.

Hacia adelante — Capítulo 09 (Arquitecturas Multiagente): la memoria compartida entre agentes introduce complejidad adicional: el olvido debe ser coordinado, las escrituras concurrentes deben ser gestionadas, y la consistencia de la memoria entre múltiples agentes es un problema de diseño distribuido. El capítulo 09 lo desarrollará desde la perspectiva de la arquitectura multiagente.

Hacia adelante — Capítulo 14 (Seguridad y Privacidad): el diseño de memoria tiene implicancias directas de privacidad: qué se guarda, durante cuánto tiempo, quién puede acceder, cómo se elimina bajo solicitud. El checklist de la sección anterior anticipa esas consideraciones; el capítulo 14 las desarrolla en el contexto regulatorio completo.

Lo que el ingeniero debería llevarse

El diseño de memoria no es un problema de almacenamiento resuelto con una base de datos vectorial. Es un problema de criterio:

- Criterio sobre qué guardar.
- Criterio sobre cuánto tiempo guardar.
- Criterio sobre qué recuperar y cuánto recuperar.
- Criterio sobre qué olvidar y cuándo.
- Criterio sobre qué controles tiene el usuario sobre su propia información.

Un sistema de memoria bien diseñado no es el que recuerda más. Es el que recuerda lo correcto, con la granularidad correcta, durante el tiempo correcto, y olvida el resto de forma limpia.

La última sección del capítulo es la autoevaluación: preguntas de comprensión y ejercicios de reflexión que permiten al lector verificar que los conceptos centrales están asentados antes de continuar.

Capítulo 04 — Sección 15

Autoevaluación y cierre

Preguntas de comprensión conceptual

Las siguientes preguntas verifican que los conceptos centrales del capítulo están asentados. Responder bien estas preguntas no requiere memorizar definiciones: requiere entender las relaciones entre los conceptos.

- 1.** Un colega propone guardar el transcript completo de cada conversación en la base de datos de memoria. ¿Cuáles son los dos problemas principales de esta decisión? ¿Qué propondrías en su lugar?
 - 2.** Tienes un sistema de asistencia para médicos. El médico menciona en una conversación que "el paciente con asma severa que vino ayer tiene contraindicación para betabloqueantes". ¿Deberías persistir esta información en memoria? ¿Por qué sí o por qué no? ¿Qué consideraciones adicionales aplican al contexto médico?
 - 3.** Explica con tus propias palabras la diferencia entre memoria episódica y memoria semántica en un sistema de IA. Da un ejemplo concreto de información que sería episódica y uno de información que sería semántica para un asistente de análisis financiero.
 - 4.** Un usuario te reporta que el sistema le está respondiendo con información de un proyecto que cerró hace cuatro meses. Diagnosifica el problema: ¿qué componente falló y cómo lo corregirías?
 - 5.** ¿Por qué la memoria semántica y RAG son complementarios y no intercambiables? Da un ejemplo de un caso de uso donde necesitarías ambos al mismo tiempo.
 - 6.** Un startup decide no implementar controles de privacidad sobre la memoria porque "están en fase de prototipo y se lo dejan para después". ¿Qué riesgos concretos introduce esta decisión? ¿Qué mínimo de control de privacidad recomendarías implementar incluso en un prototipo?
-

Ejercicios de aplicación

Ejercicio 1 — Diseño de política de retención

Estás diseñando el sistema de memoria de un asistente para equipos de recursos humanos de una empresa mediana. El sistema interactúa con cinco tipos de usuarios: recruiters, HR Business Partners, gerentes de área, empleados en proceso de onboarding, y el equipo de compensaciones.

Diseña una tabla de política de retención que especifique:

- Los tres tipos de información más importantes que merece persistir para cada rol de usuario
- El TTL sugerido para cada tipo de información
- Los criterios para actualización vs. reemplazo de la información

Ejercicio 2 — Diagnóstico de anti-patrón

Lee el siguiente fragmento de arquitectura:

"Al inicio de cada sesión, el sistema recupera los últimos 50 registros de memoria del usuario ordenados por fecha de creación y los inyecta completos en el system prompt. Durante la sesión, cada mensaje del usuario se analiza y todos los sustantivos propios se guardan como memorias nuevas. Al final de cada mes, un script manual elimina los registros más antiguos."

Identifica al menos tres anti-patrones en este diseño. Para cada uno, explica el problema y propone la corrección.

Ejercicio 3 — Extensión del laboratorio

Extiende el código del laboratorio (sección 12) para implementar el Ejercicio 1 del checklist de extensión: TTL automático. Define TTLs distintos para los cuatro tipos de memoria (preferencia, trabajo, decisión, restricción). Verifica que el filtrado funciona creando memorias con fecha de creación manual en el pasado.

Ejercicio 4 — Caso de diseño

Eres el ingeniero de IA en una plataforma de e-learning que usa un tutor conversacional de IA. El tutor trabaja con estudiantes universitarios en cursos de matemáticas, programación y estadística. Los estudiantes interactúan con el tutor múltiples veces por semana durante un semestre.

Diseña la arquitectura de memoria del tutor respondiendo:

1. ¿Qué tipos de memoria (episódica, semántica, procedimental) necesita el tutor?
 2. ¿Qué información específica debería persistir sobre cada estudiante?
 3. ¿Qué información no debería persistir nunca?
 4. ¿Qué TTL aplicarías a cada categoría?
 5. ¿Qué backend de almacenamiento elegiría para cada tipo y por qué?
 6. ¿Qué controles de privacidad son especialmente importantes dado que los usuarios son estudiantes (potencialmente menores de edad)?
-

Cierre del capítulo

La memoria transformó los sistemas de IA de herramientas de sesión única en colaboradores con continuidad. Esa transformación no ocurrió sola: ocurrió porque ingenieros diseñaron sistemas explícitos de captura, almacenamiento, recuperación y olvido.

Lo que este capítulo establece es que ese diseño es una disciplina con principios propios. No basta con conectar un LLM a una base de datos. Hay que decidir qué recordar, cómo organizarlo, qué recuperar en cada momento, durante cuánto tiempo mantenerlo activo, y cómo garantizar que el usuario tenga control sobre su propia información en el sistema.

Un sistema de memoria bien diseñado es invisible para el usuario: simplemente experimenta que el asistente lo conoce, lo entiende y no le hace repetir las mismas cosas una y otra vez. Un sistema de memoria mal diseñado es frustrante de formas específicas y predecibles: olvida lo importante, recuerda lo trivial, mezcla contextos, o no sabe olvidar cuando debería.

La diferencia entre ambos es el criterio del ingeniero que lo diseñó.

En el próximo capítulo extenderemos la arquitectura de contexto hacia el diseño de herramientas y function calling: cómo los sistemas de IA no solo recuerdan, sino que actúan sobre el mundo externo de forma controlada y verificable.

Conceptos clave del capítulo

Concepto	Definición en una línea
Memoria de trabajo	Ventana de contexto activa del modelo en la sesión actual
Memoria episódica	Registro de eventos e interacciones pasadas, situados en el tiempo
Memoria semántica	Conocimiento factual destilado sobre el usuario y el dominio
Memoria procedimental	Instrucciones y flujos que definen cómo actúa el sistema
Context Assembly	Proceso de seleccionar y priorizar qué memoria inyectar en el contexto
Upsert semántico	Insertar o actualizar una memoria según similitud con memorias existentes
TTL	Time To Live: tiempo durante el cual un registro de memoria es válido
Consolidación	Comprimir memorias episódicas acumuladas en memorias semánticas
Olvido deliberado	Política explícita de qué eliminar, cuándo y con qué criterio
Memory Store	Patrón que encapsula el acceso al backend de almacenamiento de memoria